Java 8 and Beyond - Functional Programming

#	Scenario Title	Level	Target Role
1	Sum copiesSold per category	L1	Java Backend / Analyst
2	Find longest trip per driver	L1	Java Backend
3	Total calories consumed per customer	L1	Backend / Data Analyst
4	Find single most rented movie	L1	Java Backend
5	Average study hours per student	L1	Backend / Full Stack
6	Highest single-match score per player	L1	Java Backend
7	Sort orders by total amount	L1	Backend
8	Total delivery revenue per city	L1	Java Backend
9	Sum total rental hours per bike type	L1	Java Backend
10	Total bill per booking (sum ticket prices)	L1	Java Backend
11	Distinct shows watched per viewer	L2	Backend
12	Average trip distance per driver	L1	Backend
13	Personal-best submission per coder	L1	Java Backend
14	Total units sold per seller	L1	Java Backend
15	Total reward points per member	L1	Backend
16	Total shipment weight per city	L1	Backend
17	Average pay per subject (instructors)	L2	Backend / Data Analyst
18	Restaurants with avg rating > 4.0	L2	Backend
19	Count issues by status	L1	Backend
20	Filter out books below threshold	L1	Backend / Full Stack
21	Sort players by matches played	L1	Backend
22	Detect duplicate review comments	L2	Backend / QA
23	Group review scores under each show	L1	Backend
24	Extract unique list of artists	L1	Backend / Data
25	Total calories per meal type	L1	Backend

26	Find most expensive artwork	L1	Backend
27	Sort applicants by test score	L1	Backend / HR
28	Max calories burned per user	L2	Backend
29	Filter employees by work anniversary	L1	Backend / HR
30	Average star rating per app	L1	Backend
31	List handles scoring above cutoff	L1	Backend / Data
32	Sum fares after tax (above threshold)	L1	Backend
33	Total subscription revenue per plan	L1	Backend / Finance
34	Heaviest product per category	L2	Backend
35	Apply discount on books > ₹500	L2	Backend / Full Stack
36	Design Notifier interface with default/static methods	L3	Backend / Architect
37	Sort products by category then price	L2	Backend
38	Run Runnable vs Callable jobs (log time)	L3	Backend / Concurrency
39	Total salaries of staff > ₹40L	L1	Backend / Finance
40	Use Optional for user log/display name	L1	Backend
41	Convert product names → Product objects	L2	Backend / Java Dev
42	Flatten unique list of course tags	L2	Backend
43	Compute min and max in one pass	L2	Backend
44	Sum millions of sales with parallel streams	L3	Backend / Performance
45	Partition customers (premium vs regular)	L2	Backend
46	Return immutable top 3 scores	L2	Backend
47	Build alphabetical CSV line of products	L3	Backend
48	Sort employees by dept (nulls last), then name	L2	Backend / HR
49	Safely get city via Optional chaining	L2	Backend / Full Stack
50	Optional logging for user display name	L1	Backend

51	Optional extract city from Order→Customer→Address	L2	Backend
52	Construct Product objects via constructor reference	L2	Backend
53	Unique, sorted list of tags	L2	Backend
54	Compute min/max via custom reduce	L3	Backend
55	Total sales with parallel streams	L3	Backend
56	Partition customers via partitioningBy	L2	Backend
57	Immutable top 3 scores	L2	Backend
58	Alphabetical CSV via custom collector	L3	Backend / Architect
59	Sort employees by dept (nulls last) & case-insensitive name	L2	Backend
60	Min/max/avg/sum/count in one pass	L2	Backend / Analyst
61	Partition premium/regular and count/avg per bucket	L3	Backend / Data
62	Nested grouping dept → team → avg salary	L3	Backend / HR
63	Aggregate qty by SKU (toMap w/ merge)	L3	Backend / Supply Chain
64	Accurate total money via BigDecimal	L3	Backend / Finance
65	Count hits per endpoint at scale	L3	Backend / Performance
66	Compute min & max using teeing	L3	Backend / Architect
67	Parse CSV rows into Optionals and skip bad ones	L3	Backend / Data Pipeline
68	Top N items per category (1 pass)	L4	Backend / Architect
69	Locale-aware, case/diacritic-insensitive sort	L4	Backend / Globalization

Scenario 1: Online bookstore sales analysis

The bookstore wants to see which categories are doing well by summing up copiesSold across all books belonging to the same category.

Answer:

I'll use Java Streams with Collectors.groupingBy to group books by category and then Collectors.summingInt to total the copies sold. This avoids manual loops and makes the code concise.

Code:

```
import java.util.*;
import java.util.stream.*;

class Book {
    String category;
    int copiesSold;

    public Book(String category, int copiesSold) {
        this.category = category;
        this.copiesSold = copiesSold;
    }
}

public class BookSalesDemo {
    public static void main(String[] args) {
        List<Book> books = Arrays.asList(
            new Book("Fiction", 1200),
            new Book("Science", 800),
            new Book("Fiction", 700),
            new Book("History", 450)
        );

        Map<String, Integer> salesByCategory = books.stream()
            .collect(Collectors.groupingBy(
                b -> b.category,
                Collectors.summingInt(b -> b.copiesSold)
            ));

        System.out.println("Sales by category: " + salesByCategory);
    }
}
```

Reasoning to interviewer:

- groupingBy neatly classifies all books under their category.
- summingInt is a built-in collector that directly handles the summation logic.
- The result is a Map<String, Integer> where each key is a category and value is the total sales.
- This is cleaner than manually iterating and managing a HashMap.

Future scope:

- Sort the categories by sales to find top performers.
- Move this into a service layer to support queries like “top 3 categories this month.”
- Later, data can be streamed from a database or Kafka pipeline to update results in real-time.

Scenario 2: Ride-hailing longest trip per driver

Each trip has a driver name and distance. We want to figure out the maximum trip per driver.

Answer:

I'll group trips by driver using groupingBy and then apply maxBy with a comparator on trip distance.

Code:

```
import java.util.*;
import java.util.stream.*;

class Trip {
    String driverName;
    double tripDistance;

    public Trip(String driverName, double tripDistance) {
        this.driverName = driverName;
        this.tripDistance = tripDistance;
    }

    @Override
    public String toString() {
        return driverName + " -> " + tripDistance + " km";
    }
}

public class TripAnalysis {
    public static void main(String[] args) {
        List<Trip> trips = Arrays.asList(
            new Trip("Alice", 10.5),
            new Trip("Alice", 15.2),
            new Trip("Bob", 22.0),
            new Trip("Bob", 18.7)
        );

        Map<String, Optional<Trip>> longestTripByDriver = trips.stream()
            .collect(Collectors.groupingBy(
                t -> t.driverName,
                Collectors.maxBy(Comparator.comparingDouble(t -> t.tripDistance))
            ));

        longestTripByDriver.forEach((driver, trip) ->
            System.out.println("Longest trip by " + driver + ": " + trip.orElse(null))
        );
```

```
    }
}
```

Reasoning to interviewer:

- groupingBy groups all trips under each driver’s name.
- maxBy picks the trip with the largest distance per driver.
- Comparator.comparingDouble ensures numeric comparison on distance.
- The result is a Map with driver names as keys and their longest trip wrapped in an Optional.

Future scope:

- Instead of Optional, unwrap directly using collectingAndThen(maxBy(...), Optional::get) if we’re sure every driver has trips.
- Extend analysis to average trip length per driver for performance evaluation.
- Integrate with a reporting service for dashboards used by operations teams.

Scenario 3: Food delivery calorie tracking

Each order has a customerId and caloriesConsumed. The app wants to calculate total calories consumed by each customer to personalize fitness suggestions.

Answer:

I'll group orders by customerId and then use summingDouble to aggregate the calorie count. This avoids manual accumulation and ensures clean, declarative logic.

Code:

```
import java.util.*;
import java.util.stream.*;

class Order {
    String customerId;
    double caloriesConsumed;

    public Order(String customerId, double caloriesConsumed) {
        this.customerId = customerId;
        this.caloriesConsumed = caloriesConsumed;
    }
}

public class OrderCaloriesDemo {
    public static void main(String[] args) {
        List<Order> orders = Arrays.asList(
            new Order("C1", 600),
            new Order("C1", 750),
            new Order("C2", 500),
            new Order("C2", 850)
        );

        Map<String, Double> caloriesByCustomer = orders.stream()
            .collect(Collectors.groupingBy(
                o -> o.customerId,
                Collectors.summingDouble(o -> o.caloriesConsumed)
            ));

        System.out.println("Calories consumed per customer: " +
caloriesByCustomer);
    }
}
```

Reasoning to interviewer:

- groupingBy partitions all orders by the customerId.

- summingDouble accumulates the calorie values for each group.
- The result is a map of customerId → total calories consumed.
- This is efficient compared to looping and manually updating a map.

Future scope:

- Integrate this data with a recommendation engine to suggest healthier options.
- Extend the aggregation to track weekly or monthly calorie trends.
- Persist the results in a database for long-term health insights per customer.

Scenario 4: Movie rental trending analysis

Each movie has a name and a rental count. The platform wants to find the single most rented movie.

Answer:

I'll use stream().max() with a comparator on timesRented. This directly returns the movie with the maximum rental count.

Code:

```
import java.util.*;
import java.util.stream.*;

class Movie {
    String name;
    int timesRented;

    public Movie(String name, int timesRented) {
        this.name = name;
        this.timesRented = timesRented;
    }

    @Override
    public String toString() {
        return name + " (Rented " + timesRented + " times)";
    }
}

public class MovieTrendingDemo {
    public static void main(String[] args) {
        List<Movie> movies = Arrays.asList(
            new Movie("Inception", 320),
            new Movie("Interstellar", 410),
            new Movie("The Dark Knight", 290)
        );

        Optional<Movie> topMovie = movies.stream()
            .max(Comparator.comparingInt(m -> m.timesRented));

        System.out.println("Most rented movie: " + topMovie.orElse(null));
    }
}
```

Reasoning to interviewer:

- max evaluates all elements and picks the one with the highest rental count.

- Comparator.comparingInt provides clean integer-based comparison.
- Wrapping in Optional ensures safety in case the list is empty.
- This avoids sorting the entire list when only the maximum is required.

Future scope:

- Fetch the top N trending movies using sorted().limit(N).
- Add tie-breaking logic (e.g., alphabetically) if two movies share the same rental count.
- Stream this data into a dashboard for real-time trending updates.

Scenario 5: Online learning platform – average study time

Each enrollment stores studentId and hoursSpent. The platform wants to find the average hours a student studies across all courses, to measure engagement.

Answer:

I'll use groupingBy with averagingDouble so that all records for the same student are grouped and their hours averaged.

Code:

```
import java.util.*;
import java.util.stream.*;

class CourseEnrollment {
    String studentId;
    double hoursSpent;

    public CourseEnrollment(String studentId, double hoursSpent) {
        this.studentId = studentId;
        this.hoursSpent = hoursSpent;
    }
}

public class AvgStudyDemo {
    public static void main(String[] args) {
        List<CourseEnrollment> enrollments = Arrays.asList(
            new CourseEnrollment("S1", 12.5),
            new CourseEnrollment("S1", 15.0),
            new CourseEnrollment("S2", 8.0),
            new CourseEnrollment("S2", 10.5)
        );

        Map<String, Double> avgStudyTime = enrollments.stream()
            .collect(Collectors.groupingBy(
                e -> e.studentId,
                Collectors.averagingDouble(e -> e.hoursSpent)
            ));

        System.out.println("Average study hours by student: " + avgStudyTime);
    }
}
```

Reasoning to interviewer:

- groupingBy ensures all enrollments for a student are collected together.

- averagingDouble calculates the mean value of hours for each student.
- The result is a map of studentId → average study time.
- This removes the need to manually sum and divide, keeping the code clean.

Future scope:

- Extend to track average study time per course or per semester.
- Use DoubleSummaryStatistics if min, max, and total are also needed.
- Integrate with analytics to predict student engagement or dropout risk.

Scenario 6: E-sports platform – highest match score per player

Each match result stores playerId and points. The platform wants the best single-match score for every player to determine season awards.

Answer:

I'll group results by playerId, then use maxBy with a comparator on points to fetch the highest score per player.

Code:

```
import java.util.*;
import java.util.stream.*;

class MatchResult {
    String playerId;
    int points;

    public MatchResult(String playerId, int points) {
        this.playerId = playerId;
        this.points = points;
    }

    @Override
    public String toString() {
        return playerId + " scored " + points;
    }
}

public class TopScoreDemo {
    public static void main(String[] args) {
        List<MatchResult> results = Arrays.asList(
            new MatchResult("P1", 78),
            new MatchResult("P1", 96),
            new MatchResult("P2", 88),
            new MatchResult("P2", 91)
        );

        Map<String, Optional<MatchResult>> topScorePerPlayer = results.stream()
            .collect(Collectors.groupingBy(
                r -> r.playerId,
                Collectors.maxBy(Comparator.comparingInt(r -> r.points))
            ));

        topScorePerPlayer.forEach((player, best) ->
            System.out.println("Top score for " + player + ": " + best.orElse(null))
        );
```

```
    }
}
```

Reasoning to interviewer:

- groupingBy organizes all match results per player.
- maxBy ensures only the highest scoring match is selected for each player.
- Comparator.comparingInt simplifies comparing integer points.
- The result is a map of playerId → top-scoring match wrapped in an Optional.

Future scope:

- Replace Optional with direct values using collectingAndThen if data is guaranteed.
- Add average or median points per player for deeper performance analysis.
- Extend to get top N matches for highlights or leaderboard displays.

Scenario 7: Marketplace – sorting orders by amount

Each order has an order number and total amount. The analytics team wants the orders sorted by amount so they can review high-value purchases.

Answer:

I'll use the sorted method on streams with Comparator.comparingDouble to sort by the numeric amount. If descending order is needed, I can apply .reversed().

Code:

```
import java.util.*;
import java.util.stream.*;

class Order {
    String orderNo;
    double totalAmount;

    public Order(String orderNo, double totalAmount) {
        this.orderNo = orderNo;
        this.totalAmount = totalAmount;
    }

    @Override
    public String toString() {
        return orderNo + " (₹" + totalAmount + ")";
    }
}

public class OrderSortDemo {
    public static void main(String[] args) {
        List<Order> orders = Arrays.asList(
            new Order("O-101", 1299.0),
            new Order("O-102", 4999.0),
            new Order("O-103", 2199.0)
        );

        List<Order> byAmountAsc = orders.stream()
            .sorted(Comparator.comparingDouble(o -> o.totalAmount))
            .collect(Collectors.toList());

        System.out.println("Orders sorted by amount (asc): " + byAmountAsc);

        List<Order> byAmountDesc = orders.stream()
            .sorted(Comparator.comparingDouble((Order o) ->
o.totalAmount).reversed())
            .collect(Collectors.toList());
```

```
        System.out.println("Orders sorted by amount (desc): " + byAmountDesc);
    }
}
```

Reasoning to interviewer:

- sorted creates a sorted stream without modifying the original list.
- Comparator.comparingDouble is ideal for numeric fields like totalAmount.
- Using .reversed() flips the order for descending sorting.
- This approach is more expressive than manual Collections.sort with comparators.

Future scope:

- Add multi-level sorting, e.g., by amount and then by order date.
- Apply pagination after sorting to handle large order datasets.
- Integrate with dashboards for business teams to filter top orders dynamically.

Scenario 8: Food delivery – revenue by city

Each delivery stores the city and bill amount. The company wants to calculate total revenue per city.

Answer:

I'll group deliveries by city using groupingBy and then use summingDouble to calculate the total revenue for each city.

Code:

```
import java.util.*;
import java.util.stream.*;

class Delivery {
    String city;
    double billAmount;

    public Delivery(String city, double billAmount) {
        this.city = city;
        this.billAmount = billAmount;
    }
}

public class RevenueByCityDemo {
    public static void main(String[] args) {
        List<Delivery> deliveries = Arrays.asList(
            new Delivery("Mumbai", 750),
            new Delivery("Delhi", 1200),
            new Delivery("Mumbai", 600),
            new Delivery("Bangalore", 950)
        );

        Map<String, Double> revenueByCity = deliveries.stream()
            .collect(Collectors.groupingBy(
                d -> d.city,
                Collectors.summingDouble(d -> d.billAmount)
            ));

        System.out.println("Revenue by city: " + revenueByCity);
    }
}
```

Reasoning to interviewer:

- groupingBy categorizes deliveries based on city.

- summingDouble accumulates the bill amounts for each group.
- The result is a clean city-wise revenue report.
- This approach avoids writing nested loops or manual map management.

Future scope:

- Extend to calculate average order value per city for deeper insights.
- Use DoubleSummaryStatistics to get min, max, and average in a single pass.
- Push this data to a reporting tool for city-level revenue tracking.

Scenario 9: Bike-sharing – total rental hours per bike type

Each rental log has a bikeType and hoursRented. The platform wants to know how many hours each type of bike has been used, so they can optimize their fleet.

Answer:

I'll use groupingBy on the bike type and apply summingInt to add up the rental hours.

Code:

```
import java.util.*;
import java.util.stream.*;

class BikeUsage {
    String bikeType;
    int hoursRented;

    public BikeUsage(String bikeType, int hoursRented) {
        this.bikeType = bikeType;
        this.hoursRented = hoursRented;
    }
}

public class BikeUsageDemo {
    public static void main(String[] args) {
        List<BikeUsage> usageList = Arrays.asList(
            new BikeUsage("Electric", 5),
            new BikeUsage("Mountain", 3),
            new BikeUsage("Electric", 7),
            new BikeUsage("Road", 4)
        );

        Map<String, Integer> totalHoursByBike = usageList.stream()
            .collect(Collectors.groupingBy(
                u -> u.bikeType,
                Collectors.summingInt(u -> u.hoursRented)
            ));

        System.out.println("Total rental hours by bike type: " + totalHoursByBike);
    }
}
```

Reasoning to interviewer:

- groupingBy collects all rentals under their respective bike type.
- summingInt totals up the rental hours for each type.

- The result is a map of bikeType → total rental hours.
- This is more concise and less error-prone than manually iterating with a HashMap.

Future scope:

- Extend the calculation to average hours per rental for each bike type.
- Track usage trends per day or week for demand forecasting.
- Integrate with fleet allocation logic to distribute bikes better across stations.

Scenario 10: Cinema booking – total bill per booking

Each ticket has a bookingId and ticketPrice. The system wants the total price of all tickets under the same booking so it can apply discounts on bulk purchases.

Answer:

I'll group tickets by bookingId and then use summingDouble to compute the total price per booking.

Code:

```
import java.util.*;
import java.util.stream.*;

class Ticket {
    String bookingId;
    double ticketPrice;

    public Ticket(String bookingId, double ticketPrice) {
        this.bookingId = bookingId;
        this.ticketPrice = ticketPrice;
    }
}

public class TicketBillingDemo {
    public static void main(String[] args) {
        List<Ticket> tickets = Arrays.asList(
            new Ticket("B101", 300),
            new Ticket("B101", 450),
            new Ticket("B102", 200),
            new Ticket("B102", 250)
        );

        Map<String, Double> totalPriceByBooking = tickets.stream()
            .collect(Collectors.groupingBy(
                t -> t.bookingId,
                Collectors.summingDouble(t -> t.ticketPrice)
            ));

        System.out.println("Total price per booking: " + totalPriceByBooking);
    }
}
```

Reasoning to interviewer:

- groupingBy aggregates all tickets belonging to the same booking.

- summingDouble adds up ticket prices to produce the booking total.
- The result is a map of bookingId → total bill amount.
- This approach avoids nested loops and gives a clean one-liner aggregation.

Future scope:

- Add logic to apply discounts automatically if a booking crosses a threshold.
- Extend to calculate total revenue per day or per movie show.
- Use DoubleSummaryStatistics if additional metrics like min and max ticket price are needed per booking.

Scenario 11: Video streaming – unique shows per viewer

Each viewing event logs a viewerId and a showTitle. The platform wants to know which distinct shows each user has watched, ignoring duplicates.

Answer:

I'll group events by viewerId and then use mapping with toSet so that duplicates are automatically filtered out.

Code:

```
import java.util.*;
import java.util.stream.*;

class WatchEvent {
    String viewerId;
    String showTitle;

    public WatchEvent(String viewerId, String showTitle) {
        this.viewerId = viewerId;
        this.showTitle = showTitle;
    }
}

public class UniqueShowsDemo {
    public static void main(String[] args) {
        List<WatchEvent> events = Arrays.asList(
            new WatchEvent("V1", "Breaking Bad"),
            new WatchEvent("V1", "Dark"),
            new WatchEvent("V1", "Breaking Bad"), // duplicate
            new WatchEvent("V2", "Dark"),
            new WatchEvent("V2", "Money Heist")
        );

        Map<String, Set<String>> uniqueShowsByViewer = events.stream()
            .collect(Collectors.groupingBy(
                e -> e.viewerId,
                Collectors.mapping(e -> e.showTitle, Collectors.toSet())
            ));

        System.out.println("Unique shows watched by each viewer: " +
uniqueShowsByViewer);
    }
}
```

Reasoning to interviewer:

- groupingBy ensures all watch events are grouped under each viewer.
- mapping extracts just the showTitle from each event.
- toSet automatically removes duplicates, so we don’t need extra checks.
- This results in a clean map of viewerId → unique set of show titles.

Future scope:

- Convert the set to a list if order matters (e.g., recent shows first).
- Extend the grouping to also count how many unique shows each user has watched.
- Feed this into a recommendation engine to suggest new shows based on viewing history.

Scenario 12: Ride-sharing – average trip distance per driver

Each trip log records driverId and distance in kilometers. The company wants to know the average distance of trips for each driver.

Answer:

I'll group trips by driverId and then apply averagingInt on the distance field to compute the mean value.

Code:

```
import java.util.*;
import java.util.stream.*;

class TripLog {
    String driverId;
    int distanceKm;

    public TripLog(String driverId, int distanceKm) {
        this.driverId = driverId;
        this.distanceKm = distanceKm;
    }
}

public class AvgTripDistanceDemo {
    public static void main(String[] args) {
        List<TripLog> logs = Arrays.asList(
            new TripLog("D1", 12),
            new TripLog("D1", 18),
            new TripLog("D2", 25),
            new TripLog("D2", 15)
        );

        Map<String, Double> avgDistanceByDriver = logs.stream()
            .collect(Collectors.groupingBy(
                t -> t.driverId,
                Collectors.averagingInt(t -> t.distanceKm)
            ));

        System.out.println("Average trip distance per driver: " +
avgDistanceByDriver);
    }
}
```

Reasoning to interviewer:

- groupingBy organizes all trips under the corresponding driverId.

- averagingInt automatically calculates the mean distance for each driver.
- The output is a map of driverId → average trip distance.
- This avoids manual sum/count bookkeeping and keeps the code concise.

Future scope:

- Switch to averagingDouble if distances can include decimals.
- Combine with summarizingInt to also get min, max, and total distance in one pass.
- Use the averages to benchmark drivers and suggest route optimizations.

Scenario 13: Coding challenge – personal best per coder

Each submission has a coderId and points. The platform wants to show each coder's best score across all their submissions.

Answer:

I'll group submissions by coderId and then use maxBy with a comparator on points to pick the top-scoring submission for each coder.

Code:

```
import java.util.*;
import java.util.stream.*;

class Submission {
    String coderId;
    int points;

    public Submission(String coderId, int points) {
        this.coderId = coderId;
        this.points = points;
    }

    @Override
    public String toString() {
        return coderId + " -> " + points + " pts";
    }
}

public class BestScoreDemo {
    public static void main(String[] args) {
        List<Submission> submissions = Arrays.asList(
            new Submission("C1", 1200),
            new Submission("C1", 1450),
            new Submission("C2", 980),
            new Submission("C2", 1310)
        );

        Map<String, Optional<Submission>> bestByCoder = submissions.stream()
            .collect(Collectors.groupingBy(
                s -> s.coderId,
                Collectors.maxBy(Comparator.comparingInt(s -> s.points))
            ));

        bestByCoder.forEach((coder, best) ->
            System.out.println("Best for " + coder + ": " + best.orElse(null))
        );
```

```
    }
}
```

Reasoning to interviewer:

- groupingBy collects all submissions under each coderId.
- maxBy ensures we only keep the highest points per coder.
- Comparator.comparingInt provides clean numeric comparison on the points field.
- The result is a map of coderId → best submission wrapped in an Optional.

Future scope:

- Replace Optional with direct values using collectingAndThen(maxBy(...), Optional::get) if every coder is guaranteed at least one submission.
- Track not only the best but also the average and total points per coder for leaderboards.
- Integrate with ranking logic to highlight top coders globally.

Scenario 14: Marketplace – total units sold per seller

Each sale record has a seller name and number of units sold. The marketplace wants to know total units per seller to spot top performers.

Answer:

I'll group sales by sellerName and apply summingInt to add up all units sold per seller.

Code:

```
import java.util.*;
import java.util.stream.*;

class Sale {
    String sellerName;
    int unitsSold;

    public Sale(String sellerName, int unitsSold) {
        this.sellerName = sellerName;
        this.unitsSold = unitsSold;
    }
}

public class SellerPerformanceDemo {
    public static void main(String[] args) {
        List<Sale> sales = Arrays.asList(
            new Sale("Alice", 320),
            new Sale("Alice", 180),
            new Sale("Bob", 500),
            new Sale("Bob", 140)
        );

        Map<String, Integer> totalUnitsBySeller = sales.stream()
            .collect(Collectors.groupingBy(
                s -> s.sellerName,
                Collectors.summingInt(s -> s.unitsSold)
            ));

        System.out.println("Total units by seller: " + totalUnitsBySeller);
    }
}
```

Reasoning to interviewer:

- groupingBy aggregates all sale records under each seller's name.

- summingInt efficiently totals the units for each seller.
- The result is a map of sellerName → total units sold.
- This avoids manual loops and conditionals for accumulation.

Future scope:

- Rank sellers by total units sold to identify top performers.
- Extend the logic to compute revenue per seller if product price is available.
- Use this data to design incentive programs for high-performing sellers.

Scenario 15: Fitness app – total reward points per member

Each reward entry has a memberId and rewardPoints. The company wants to calculate total points each member has earned to track loyalty.

Answer:

I'll group reward entries by memberId and apply summingInt to add up all points per member.

Code:

```
import java.util.*;
import java.util.stream.*;

class RewardEntry {
    String memberId;
    int rewardPoints;

    public RewardEntry(String memberId, int rewardPoints) {
        this.memberId = memberId;
        this.rewardPoints = rewardPoints;
    }
}

public class RewardPointsDemo {
    public static void main(String[] args) {
        List<RewardEntry> rewards = Arrays.asList(
            new RewardEntry("M1", 120),
            new RewardEntry("M1", 80),
            new RewardEntry("M2", 200),
            new RewardEntry("M2", 150)
        );

        Map<String, Integer> totalPointsByMember = rewards.stream()
            .collect(Collectors.groupingBy(
                r -> r.memberId,
                Collectors.summingInt(r -> r.rewardPoints)
            ));

        System.out.println("Total reward points by member: " +
totalPointsByMember);
    }
}
```

Reasoning to interviewer:

- groupingBy organizes all reward entries under the corresponding member.

- summingInt efficiently accumulates integer points for each member.
- The result is a clean map of memberId → total reward points.
- This avoids writing boilerplate loops and makes the aggregation concise.

Future scope:

- Extend logic to calculate average reward points per transaction for each member.
- Integrate thresholds (e.g., gold, silver tiers) based on cumulative points.
- Persist totals to a database for loyalty program dashboards.

Scenario 16: Warehouse – total shipment weight per city

Each shipment stores a city and load weight. The warehouse wants to know the total shipped weight for each city to monitor distribution.

Answer:

I'll group shipments by city and apply summingDouble to total the load weights.

Code:

```
import java.util.*;
import java.util.stream.*;

class Shipment {
    String city;
    double loadWeight;

    public Shipment(String city, double loadWeight) {
        this.city = city;
        this.loadWeight = loadWeight;
    }
}

public class ShipmentWeightDemo {
    public static void main(String[] args) {
        List<Shipment> shipments = Arrays.asList(
            new Shipment("Delhi", 12.5),
            new Shipment("Mumbai", 18.0),
            new Shipment("Delhi", 9.5),
            new Shipment("Chennai", 6.0)
        );

        Map<String, Double> totalWeightByCity = shipments.stream()
            .collect(Collectors.groupingBy(
                s -> s.city,
                Collectors.summingDouble(s -> s.loadWeight)
            ));

        System.out.println("Total shipment weight per city: " + totalWeightByCity);
    }
}
```

Reasoning to interviewer:

- groupingBy clusters all shipments by city.
- summingDouble accumulates weights since load values are decimal.

- The result is a map of city → total shipped weight.
- This approach avoids manual iteration and makes aggregation straightforward.

Future scope:

- Extend the calculation to average load weight per shipment in each city.
- Use DoubleSummaryStatistics if min, max, and count are also required.
- Feed this data into a logistics system for optimizing truck allocation per region.

Scenario 17: Online university – average pay per subject

Each instructor record has a subject and an annual salary. The university wants to know the average pay for each subject to evaluate salary structures.

Answer:

I'll group instructors by subject and then apply averagingDouble on annualPay.

Code:

```
import java.util.*;
import java.util.stream.*;

class Instructor {
    String subject;
    double annualPay;

    public Instructor(String subject, double annualPay) {
        this.subject = subject;
        this.annualPay = annualPay;
    }
}

public class InstructorPayDemo {
    public static void main(String[] args) {
        List<Instructor> instructors = Arrays.asList(
            new Instructor("Computer Science", 90000),
            new Instructor("Mathematics", 70000),
            new Instructor("Computer Science", 85000),
            new Instructor("Physics", 75000)
        );

        Map<String, Double> avgPayBySubject = instructors.stream()
            .collect(Collectors.groupingBy(
                i -> i.subject,
                Collectors.averagingDouble(i -> i.annualPay)
            ));

        System.out.println("Average pay by subject: " + avgPayBySubject);
    }
}
```

Reasoning to interviewer:

- groupingBy clusters instructors based on their subject.
- averagingDouble directly computes the mean salary for each group.

- The output is a subject → average salary map.
- This approach eliminates manual summation and division logic.

Future scope:

- Use DoubleSummaryStatistics to also capture min, max, and total salaries in one pass.
- Compare average pay across subjects for equity analysis.
- Extend the grouping to department level for broader reports.

Scenario 18: Food delivery – top-rated restaurants

Each review has a restaurantId, userId, and a star rating. The company wants to identify restaurants with an average rating above 4.0.

Answer:

I'll group reviews by restaurantId, compute average stars with averagingDouble, and then filter restaurants where the average exceeds 4.0.

Code:

```
import java.util.*;
import java.util.stream.*;

class Review {
    String restaurantId;
    String userId;
    double stars;

    public Review(String restaurantId, String userId, double stars) {
        this.restaurantId = restaurantId;
        this.userId = userId;
        this.stars = stars;
    }
}

public class TopRatedRestaurantsDemo {
    public static void main(String[] args) {
        List<Review> reviews = Arrays.asList(
            new Review("R1", "U1", 4.5),
            new Review("R1", "U2", 4.2),
            new Review("R2", "U3", 3.8),
            new Review("R2", "U4", 4.1),
            new Review("R3", "U5", 5.0)
        );

        Map<String, Double> topRatedRestaurants = reviews.stream()
            .collect(Collectors.groupingBy(
                r -> r.restaurantId,
                Collectors.averagingDouble(r -> r.stars)
            ))
            .entrySet().stream()
            .filter(e -> e.getValue() > 4.0)
            .collect(Collectors.toMap(Map.Entry::getKey, Map.Entry::getValue));

        System.out.println("Highly-rated restaurants: " + topRatedRestaurants);
    }
```

```
}
```

Reasoning to interviewer:

- groupingBy aggregates reviews by restaurantId.
- averagingDouble calculates the mean rating for each restaurant.
- A second stream filters only those with an average above 4.0.
- The result is a map of restaurantId → average rating for highly rated places.

Future scope:

- Add sorting by rating to display top N restaurants.
- Combine this with the number of reviews to avoid bias from single ratings.
- Extend to compute city-wise or cuisine-wise top performers.

Scenario 19: Bug-tracking system – issues by status

Each issue record has a title, severity, and status. The team wants to count how many issues are in each status category (Open, Resolved, Closed, etc.).

Answer:

I'll group issues by status and then apply counting to tally the number of issues in each category.

Code:

```
import java.util.*;
import java.util.stream.*;

class Issue {
    String title;
    String severity;
    String status;

    public Issue(String title, String severity, String status) {
        this.title = title;
        this.severity = severity;
        this.status = status;
    }
}

public class IssueStatusReport {
    public static void main(String[] args) {
        List<Issue> issues = Arrays.asList(
            new Issue("Login Bug", "High", "Open"),
            new Issue("UI Glitch", "Low", "Resolved"),
            new Issue("Payment Error", "Critical", "Open"),
            new Issue("Notification Delay", "Medium", "Resolved")
        );

        Map<String, Long> issueCountByStatus = issues.stream()
            .collect(Collectors.groupingBy(
                i -> i.status,
                Collectors.counting()
            ));

        System.out.println("Issue count by status: " + issueCountByStatus);
    }
}
```

Reasoning to interviewer:

- groupingBy groups all issues based on their status.
- counting tallies how many issues are in each group.
- The result is a map of status → issue count.
- This is much simpler than manually iterating and incrementing counters.

Future scope:

- Extend grouping by severity as well (nested grouping: status → severity → count).
- Filter only “Open” issues for live dashboard tracking.
- Combine with SLA metrics to show how long issues stay in each status.

Scenario 20: Library – filtering books below circulation threshold

Each stock record has a title and number of available copies. The librarian wants to exclude books with fewer copies than the minimum required.

Answer:

I'll use the filter method on the stream to keep only those books where copiesAvailable is greater than the threshold.

Code:

```
import java.util.*;
import java.util.stream.*;

class StockItem {
    String title;
    int copiesAvailable;

    public StockItem(String title, int copiesAvailable) {
        this.title = title;
        this.copiesAvailable = copiesAvailable;
    }

    @Override
    public String toString() {
        return title + " (Copies: " + copiesAvailable + ")";
    }
}

public class StockFilterDemo {
    public static void main(String[] args) {
        List<StockItem> items = Arrays.asList(
            new StockItem("Algorithms", 3),
            new StockItem("Databases", 12),
            new StockItem("Networking", 1)
        );

        int minRequired = 5;

        List<StockItem> available = items.stream()
            .filter(i -> i.copiesAvailable > minRequired)
            .collect(Collectors.toList());

        System.out.println("Books meeting circulation threshold: " + available);
    }
}
```

Reasoning to interviewer:

- filter directly removes items that don’t meet the threshold condition.
- This makes the intent very clear compared to using loops with conditionals.
- The output is a list of stock items that have enough copies for circulation.

Future scope:

- Parameterize the threshold based on category (e.g., reference books vs fiction).
- Chain more conditions, like filtering only books available in a certain section.
- Streamline into a monitoring system that flags low-stock titles for restocking.

Scenario 21: Sports league – sorting players by matches played

Each player record has a name and matches played. The app wants to display players ordered by the number of matches.

Answer:

I'll use the sorted method with Comparator.comparingInt on the matchesPlayed field.

Code:

```
import java.util.*;
import java.util.stream.*;

class Player {
    String name;
    int matchesPlayed;

    public Player(String name, int matchesPlayed) {
        this.name = name;
        this.matchesPlayed = matchesPlayed;
    }

    @Override
    public String toString() {
        return name + " (Matches: " + matchesPlayed + ")";
    }
}

public class PlayerSortDemo {
    public static void main(String[] args) {
        List<Player> players = Arrays.asList(
            new Player("Rahul", 45),
            new Player("Amit", 28),
            new Player("Sneha", 63)
        );

        List<Player> sortedPlayers = players.stream()
            .sorted(Comparator.comparingInt(p -> p.matchesPlayed))
            .collect(Collectors.toList());

        System.out.println("Players sorted by matches played: " + sortedPlayers);
    }
}
```

Reasoning to interviewer:

- sorted arranges stream elements without mutating the original list.

- Comparator.comparingInt provides a clean way to compare the integer field matchesPlayed.
- The result is an ordered list of players based on their experience.

Future scope:

- Add .reversed() if descending order (most matches first) is required.
- Apply secondary sorting by player name if two players have the same matches count.
- Use pagination or limits when the player list is very large.

Scenario 22: Review portal – detecting duplicate comments

Each review stores a userId and a comment. The company wants to identify duplicate comments submitted by different users to flag spammy behavior.

Answer:

I'll group reviews by their comment text, and then filter out only those groups where more than one review exists.

Code:

```
import java.util.*;
import java.util.stream.*;

class Review {
    String userId;
    String comment;

    public Review(String userId, String comment) {
        this.userId = userId;
        this.comment = comment;
    }

    @Override
    public String toString() {
        return comment + " (User: " + userId + ")";
    }
}

public class DuplicateReviewsDemo {
    public static void main(String[] args) {
        List<Review> reviews = Arrays.asList(
            new Review("U1", "Excellent product"),
            new Review("U2", "Fast delivery"),
            new Review("U3", "Excellent product")
        );

        Map<String, List<Review>> groupedReviews = reviews.stream()
            .collect(Collectors.groupingBy(r -> r.comment));

        List<String> duplicateComments = groupedReviews.entrySet().stream()
            .filter(e -> e.getValue().size() > 1)
            .map(Map.Entry::getKey)
            .collect(Collectors.toList());

        System.out.println("Duplicate review comments: " + duplicateComments);
    }
```

```
}
```

Reasoning to interviewer:

- groupingBy clusters all reviews based on the exact comment text.
- Groups with more than one review indicate duplicates.
- Filtering by size() > 1 ensures we only keep repeated comments.
- This makes it easy to detect spam submissions across different users.

Future scope:

- Normalize comments by trimming whitespace or ignoring case before grouping.
- Integrate NLP to detect near-duplicate reviews (e.g., “Great product” vs “Product is great”).
- Automatically flag users who repeatedly submit spam comments.

Scenario 23: Streaming platform – grouping scores by show

Each review has a showId and a score. The company wants to group all review scores under each show to prepare performance reports.

Answer:

I'll group reviews by showId and then use mapping to collect only the scores into a list.

Code:

```
import java.util.*;
import java.util.stream.*;

class Review {
    String showId;
    double score;

    public Review(String showId, double score) {
        this.showId = showId;
        this.score = score;
    }
}

public class ScoresByShowDemo {
    public static void main(String[] args) {
        List<Review> reviews = Arrays.asList(
            new Review("S1", 4.8),
            new Review("S1", 3.5),
            new Review("S2", 5.0),
            new Review("S2", 4.6)
        );

        Map<String, List<Double>> scoresByShow = reviews.stream()
            .collect(Collectors.groupingBy(
                r -> r.showId,
                Collectors.mapping(r -> r.score, Collectors.toList())
            ));

        System.out.println("Scores by show: " + scoresByShow);
    }
}
```

Reasoning to interviewer:

- groupingBy collects reviews under each showId.

- mapping extracts the score field from each review.
- toList preserves all scores as a list for that show.
- This allows further analysis, like calculating averages or distributions.

Future scope:

- Replace toList with averagingDouble to compute average ratings directly.
- Add filtering to exclude shows with very few ratings.
- Extend the report to highlight top 5 shows based on average score.

Scenario 24: Music library – extracting unique artists

Each track has a title and an artist. The library wants to create a unique list of all artists to avoid duplication in reports.

Answer:

I'll use map to extract the artist field from each track, and then apply distinct to eliminate duplicates.

Code:

```
import java.util.*;
import java.util.stream.*;

class Track {
    String title;
    String artist;

    public Track(String title, String artist) {
        this.title = title;
        this.artist = artist;
    }
}

public class UniqueArtistsDemo {
    public static void main(String[] args) {
        List<Track> tracks = Arrays.asList(
            new Track("Shape of You", "Ed Sheeran"),
            new Track("Perfect", "Ed Sheeran"),
            new Track("Blinding Lights", "The Weeknd")
        );

        List<String> uniqueArtists = tracks.stream()
            .map(t -> t.artist)
            .distinct()
            .collect(Collectors.toList());

        System.out.println("Unique artists: " + uniqueArtists);
    }
}
```

Reasoning to interviewer:

- map extracts the artist from each track object.
- distinct ensures duplicate artist names are removed.
- The final result is a clean list of unique artists in the library.

Future scope:

- Sort the list alphabetically before collecting.
- Convert to a Set if uniqueness is more important than ordering.
- Add case-insensitive comparison for detecting duplicates like "Ed Sheeran" vs "ed sheeran".

Scenario 25: Fitness app – total calories per meal type

Each meal log has a mealType and calories. The app wants to calculate how many calories were consumed for each type of meal.

Answer:

I'll group meals by mealType and apply summingDouble to total calories.

Code:

```
import java.util.*;
import java.util.stream.*;

class Meal {
    String mealType;
    double calories;

    public Meal(String mealType, double calories) {
        this.mealType = mealType;
        this.calories = calories;
    }
}

public class CaloriesByMealDemo {
    public static void main(String[] args) {
        List<Meal> meals = Arrays.asList(
            new Meal("Breakfast", 350.0),
            new Meal("Lunch", 600.0),
            new Meal("Breakfast", 250.0)
        );

        Map<String, Double> caloriesByMeal = meals.stream()
            .collect(Collectors.groupingBy(
                m -> m.mealType,
                Collectors.summingDouble(m -> m.calories)
            ));

        System.out.println("Total calories by meal type: " + caloriesByMeal);
    }
}
```

Reasoning to interviewer:

- groupingBy groups meals under each mealType.
- summingDouble totals up the calories for each group.
- The result is a map of mealType → total calories consumed.

Future scope:

- Extend to compute average calories per meal type with averagingDouble.
- Use summarizingDouble if min, max, and count statistics are also required.
- Visualize the data for users to understand daily calorie distribution by meals.

Scenario 26: Art auction – most expensive artwork

Each artwork has a title and bid price. The platform wants to find the single artwork with the highest bid.

Answer:

I'll use the max method on the stream with a comparator based on bidPrice.

Code:

```
import java.util.*;
import java.util.stream.*;

class Artwork {
    String title;
    double bidPrice;

    public Artwork(String title, double bidPrice) {
        this.title = title;
        this.bidPrice = bidPrice;
    }

    @Override
    public String toString() {
        return title + " ($" + bidPrice + ")";
    }
}

public class MostExpensiveArtworkDemo {
    public static void main(String[] args) {
        List<Artwork> artworks = Arrays.asList(
            new Artwork("Mona Lisa Replica", 5000),
            new Artwork("Abstract Painting", 8200),
            new Artwork("Modern Sculpture", 4500)
        );

        Optional<Artwork> mostExpensive = artworks.stream()
            .max(Comparator.comparingDouble(a -> a.bidPrice));

        System.out.println("Most expensive artwork: " + mostExpensive.orElse(null));
    }
}
```

Reasoning to interviewer:

- max finds the highest element based on the comparator provided.
- Comparator.comparingDouble ensures numeric comparison on the bid price.

- Wrapping the result in Optional keeps the code safe in case the list is empty.
- This avoids having to sort the entire list when only the maximum is needed.

Future scope:

- Use min if the requirement is to find the cheapest artwork.
- Extend the feature to highlight the top N expensive artworks using sorted().limit(N).
- Integrate the logic with a live auction dashboard for real-time updates.

Scenario 27: Job portal – sorting applicants by score

Each applicant has a name and test score. The platform wants to sort applicants in ascending order of scores before interviews.

Answer:

I'll use the sorted method with Comparator.comparingInt on the score field.

Code:

```
import java.util.*;
import java.util.stream.*;

class Applicant {
    String name;
    int score;

    public Applicant(String name, int score) {
        this.name = name;
        this.score = score;
    }

    @Override
    public String toString() {
        return name + " (Score: " + score + ")";
    }
}

public class ApplicantSortDemo {
    public static void main(String[] args) {
        List<Applicant> applicants = Arrays.asList(
            new Applicant("Ravi", 78),
            new Applicant("Meera", 92),
            new Applicant("Aman", 85)
        );

        List<Applicant> sortedApplicants = applicants.stream()
            .sorted(Comparator.comparingInt(a -> a.score))
            .collect(Collectors.toList());

        System.out.println("Applicants sorted by score: " + sortedApplicants);
    }
}
```

Reasoning to interviewer:

- sorted arranges stream elements without altering the original list.

- Comparator.comparingInt is efficient for integer comparison.
- The output is an ordered list of applicants based on their test scores.
- This approach is concise compared to manual sorting using Collections.sort.

Future scope:

- Reverse the comparator to get descending order if the requirement is to shortlist top scorers first.
- Add secondary sorting by applicant name in case of equal scores.
- Integrate pagination to handle large applicant pools efficiently.

Scenario 28: Fitness app – highest calories burned per user

Each workout log records the user and calories burned. The app wants to find the maximum calories burned in a single session for each user.

Answer:

I'll group workout logs by user and then apply maxBy with a comparator on caloriesBurned.

Code:

```
import java.util.*;
import java.util.stream.*;

class WorkoutLog {
    String user;
    double caloriesBurned;

    public WorkoutLog(String user, double caloriesBurned) {
        this.user = user;
        this.caloriesBurned = caloriesBurned;
    }

    @Override
    public String toString() {
        return user + " (" + caloriesBurned + " cal)";
    }
}

public class MaxCaloriesDemo {
    public static void main(String[] args) {
        List<WorkoutLog> logs = Arrays.asList(
            new WorkoutLog("User1", 500),
            new WorkoutLog("User2", 650),
            new WorkoutLog("User1", 720),
            new WorkoutLog("User2", 580)
        );

        Map<String, Optional<WorkoutLog>> maxCaloriesByUser = logs.stream()
            .collect(Collectors.groupingBy(
                l -> l.user,
                Collectors.maxBy(Comparator.comparingDouble(l -> l.caloriesBurned))
            ));

        maxCaloriesByUser.forEach((user, log) ->
            System.out.println("Max calories burned by " + user + ": " + log.orElse(null))
        );
```

```
    }
}
```

Reasoning to interviewer:

- groupingBy organizes logs per user.
- maxBy ensures only the highest-calorie workout per user is kept.
- Comparator.comparingDouble provides a clean numeric comparison.
- Wrapping in Optional avoids null issues if a user has no workouts.

Future scope:

- Replace Optional with direct values using collectingAndThen if every user is guaranteed to have at least one log.
- Track not just max, but also min/avg calories burned using summarizingDouble.
- Integrate this into leaderboards showing top workouts per week.

Scenario 29: HR portal – filtering work anniversaries

Each employee record has a full name and date of joining. The company wants to find employees whose work anniversary falls on a given month/day, ignoring the year.

Answer:

I'll use Java 8's MonthDay to match month and day, then filter employees accordingly.

Code:

```
import java.time.LocalDate;
import java.time.MonthDay;
import java.util.*;
import java.util.stream.*;

class Employee {
    String fullName;
    LocalDate dateOfJoining;

    public Employee(String fullName, LocalDate dateOfJoining) {
        this.fullName = fullName;
        this.dateOfJoining = dateOfJoining;
    }

    @Override
    public String toString() {
        return fullName + " (Joined: " + dateOfJoining + ")";
    }
}

public class AnniversaryFilterDemo {
    public static void main(String[] args) {
        List<Employee> staff = Arrays.asList(
            new Employee("Neha", LocalDate.of(2017, 3, 10)),
            new Employee("Arjun", LocalDate.of(2019, 3, 10)),
            new Employee("Sana", LocalDate.of(2021, 7, 25))
        );

        int month = 3, day = 10;
        MonthDay target = MonthDay.of(month, day);

        List<Employee> anniversaries = staff.stream()
            .filter(e -> MonthDay.from(e.dateOfJoining).equals(target))
            .collect(Collectors.toList());
```

```
        System.out.println("Anniversaries on " + month + "/" + day + ": " +
anniversaries);
    }
}
```

Reasoning to interviewer:

- MonthDay helps isolate month/day, ignoring the year part of the LocalDate.
- filter checks whether each employee's joining date matches the target.
- The result is a list of employees who share the same anniversary date, regardless of joining year.

Future scope:

- Automate sending congratulatory emails to filtered employees.
- Extend logic to check anniversaries within a week (not just exact day).
- Use this approach for birthdays or other recurring annual events too.

Scenario 30: App store – average rating per app

Each review has an appId, star rating, and note. The app store wants to calculate the average star rating per app to power the rankings page.

Answer:

I'll group reviews by appId and apply averagingDouble to compute the average stars for each app.

Code:

```
import java.util.*;
import java.util.stream.*;

class AppReview {
    String appId;
    double stars;
    String note;

    public AppReview(String appId, double stars, String note) {
        this.appId = appId;
        this.stars = stars;
        this.note = note;
    }
}

public class AvgAppRatingDemo {
    public static void main(String[] args) {
        List<AppReview> allReviews = Arrays.asList(
            new AppReview("A1", 4.7, "Smooth UI"),
            new AppReview("A1", 3.9, "Good but buggy"),
            new AppReview("A2", 5.0, "Perfect"),
            new AppReview("A2", 4.2, "Nice features"),
            new AppReview("A3", 3.5, "Average")
        );

        Map<String, Double> avgStarsByApp = allReviews.stream()
            .collect(Collectors.groupingBy(
                r -> r.appId,
                Collectors.averagingDouble(r -> r.stars)
            ));

        System.out.println("Average rating by app: " + avgStarsByApp);
    }
}
```

Reasoning to interviewer:

- groupingBy clusters reviews based on appId.
- averagingDouble calculates the mean star rating for each group.
- This produces a map of appId → average rating.
- The solution is clean and avoids manual sum/count bookkeeping.

Future scope:

- Sort apps by average rating to display rankings.
- Add filtering to ignore apps with very few reviews.
- Use DoubleSummaryStatistics to also track min, max, and count per app.

Scenario 31: Coding contest – filtering by score cutoff

Each contestant has a handle and points. The platform wants to list the handles of those who scored above a given cutoff.

Answer:

I'll filter contestants based on their score and then use map to extract just the handles.

Code:

```
import java.util.*;
import java.util.stream.*;

class Contestant {
    String handle;
    int points;

    public Contestant(String handle, int points) {
        this.handle = handle;
        this.points = points;
    }

    public String getHandle() { return handle; }
    public int getPoints() { return points; }
}

public class QualifiedContestantsDemo {
    public static void main(String[] args) {
        List<Contestant> contestants = Arrays.asList(
            new Contestant("coder_raj", 1860),
            new Contestant("byte_ninja", 2120),
            new Contestant("algoQueen", 1750)
        );

        int cutoff = 1800;

        List<String> qualifiedHandles = contestants.stream()
            .filter(c -> c.getPoints() > cutoff)
            .map(Contestant::getHandle)
            .collect(Collectors.toList());

        System.out.println("Qualified contestants (> " + cutoff + "): " +
qualifiedHandles);
    }
}
```

Reasoning to interviewer:

- filter removes contestants who didn’t meet the cutoff.
- map extracts only the handle field from the remaining contestants.
- The result is a clean list of qualified contestant handles.
- This keeps the pipeline focused and avoids extra loops.

Future scope:

- Sort the qualified contestants by score in descending order.
- Display the top N handles for leaderboard-style output.
- Extend to group handles by contest division (e.g., beginner, advanced).

Scenario 32: Ride-hailing – total fare after tax for high-value trips

Each trip stores its type and fare. Management wants to consider only trips above a threshold fare, apply tax, and calculate the total revenue.

Answer:

I'll filter trips above the threshold, then use mapToDouble to apply tax to each fare, and finally sum the results.

Code:

```
import java.util.*;
import java.util.stream.*;

class Trip {
    String rideType;
    double fare;

    public Trip(String rideType, double fare) {
        this.rideType = rideType;
        this.fare = fare;
    }
}

public class TotalFareWithTaxDemo {
    public static void main(String[] args) {
        List<Trip> trips = Arrays.asList(
            new Trip("Premium", 300),
            new Trip("Standard", 120),
            new Trip("Economy", 80)
        );

        double taxRate = 0.10;
        double minFare = 100.0;

        double totalWithTax = trips.stream()
            .filter(t -> t.fare > minFare)
            .mapToDouble(t -> t.fare * (1 + taxRate))
            .sum();

        System.out.println("Total fare (after tax): " + totalWithTax);
    }
}
```

Reasoning to interviewer:

- filter ensures we only consider trips with fare above the threshold.

- mapToDouble applies the tax formula to each eligible fare.
- sum aggregates all adjusted fares into a single total.
- This approach is clean and avoids explicit loops and accumulators.

Future scope:

- Make the tax rate dynamic based on ride type (e.g., higher for Premium).
- Extend to group by rideType and calculate taxed totals per category.
- Log detailed breakdowns (pre-tax vs post-tax) for auditing purposes.

Scenario 33: Music subscription – revenue per plan

Each subscription record contains a plan name and amount paid. The company wants to calculate the total revenue per plan.

Answer:

I'll group subscriptions by plan name and then use summingDouble to add up the payments.

Code:

```
import java.util.*;
import java.util.stream.*;

class Subscription {
    String planName;
    double amountPaid;

    public Subscription(String planName, double amountPaid) {
        this.planName = planName;
        this.amountPaid = amountPaid;
    }
}

public class RevenueByPlanDemo {
    public static void main(String[] args) {
        List<Subscription> subscriptions = Arrays.asList(
            new Subscription("Gold", 999),
            new Subscription("Gold", 799),
            new Subscription("Silver", 499),
            new Subscription("Silver", 299)
        );

        Map<String, Double> revenueByPlan = subscriptions.stream()
            .collect(Collectors.groupingBy(
                s -> s.planName,
                Collectors.summingDouble(s -> s.amountPaid)
            ));

        System.out.println("Revenue by plan: " + revenueByPlan);
    }
}
```

Reasoning to interviewer:

- groupingBy organizes subscriptions under each plan.

- summingDouble totals up the payments for each group.
- The result is a clear map of planName → total revenue.
- This avoids writing manual aggregation logic with conditionals.

Future scope:

- Sort plans by revenue to see the most profitable ones.
- Add filtering to exclude trials or promotional plans.
- Track average revenue per plan with averagingDouble.

Scenario 34: Grocery app – heaviest product per category

Each product belongs to a category and has a weight. The company wants to find the single heaviest product in every category to assist logistics planning.

Answer:

I'll group products by category and then apply maxBy with a comparator on weight.

Code:

```
import java.util.*;
import java.util.stream.*;

class Product {
    String category;
    double weight;

    public Product(String category, double weight) {
        this.category = category;
        this.weight = weight;
    }

    @Override
    public String toString() {
        return category + " item (" + weight + " kg)";
    }
}

public class HeaviestProductDemo {
    public static void main(String[] args) {
        List<Product> products = Arrays.asList(
            new Product("Fruits", 1.2),
            new Product("Fruits", 0.8),
            new Product("Vegetables", 5.0),
            new Product("Vegetables", 2.3),
            new Product("Dairy", 3.0)
        );

        Map<String, Optional<Product>> heaviestByCategory = products.stream()
            .collect(Collectors.groupingBy(
                p -> p.category,
                Collectors.maxBy(Comparator.comparingDouble(p -> p.weight))
            ));

        heaviestByCategory.forEach((cat, prod) ->
            System.out.println("Heaviest in " + cat + ": " + prod.orElse(null))
        );
    }
```

```
}
```

Reasoning to interviewer:

- groupingBy ensures products are categorized (Fruits, Vegetables, etc.).
- maxBy finds the heaviest product in each category.
- Comparator.comparingDouble cleanly compares weights.
- Optional keeps the solution safe in case a category has no items.

Future scope:

- Replace Optional with direct values using collectingAndThen.
- Extend to show top N heaviest products per category instead of only the heaviest.
- Integrate with logistics planning to optimize truck loads by category weight distribution.

Scenario 35: Bookstore – apply discount to expensive books

The cart contains books with title and price. Management wants to apply a discount only to books priced above ₹500, while showing both the original and discounted prices.

Answer:

I'll filter books above ₹500, use peek to log the original prices, then use map to apply the discount and finally print the discounted prices.

Code:

```
import java.util.*;
import java.util.stream.*;

class Book {
    String title;
    double price;

    public Book(String title, double price) {
        this.title = title;
        this.price = price;
    }

    public double getPrice() { return price; }
    public void setPrice(double price) { this.price = price; }

    @Override
    public String toString() {
        return title + " : ₹" + price;
    }
}

public class DiscountedBooksDemo {
    public static void main(String[] args) {
        List<Book> cart = Arrays.asList(
            new Book("System Design Guide", 1200),
            new Book("Java Essentials", 450),
            new Book("Spring Boot in Depth", 800)
        );

        double discountRate = 0.15;

        cart.stream()
            .filter(b -> b.getPrice() > 500)
            .peek(b -> System.out.println("Original price of " + b.title + ": ₹" +
b.getPrice()))
```

```
            .map(b -> {
                b.setPrice(b.getPrice() * (1 - discountRate));
                return b;
            })
            .forEach(b -> System.out.println("Discounted price of " + b.title + ": ₹" +
b.getPrice()));
    }
}
```

Reasoning to interviewer:

- filter ensures discounts apply only to books above ₹500.
- peek provides a side-effect step to log original prices without breaking the stream pipeline.
- map updates each book's price with the discount formula.
- forEach prints the final discounted price for clarity.

Future scope:

- Apply different discount rates depending on the category or publisher.
- Move discount logic into a dedicated method for reusability.
- Generate a detailed bill showing total savings across all discounted items.

Scenario 36: Notification service – reusable interface with default and static methods

The service needs a common contract for different notification types (Push, Email, SMS). The goal is to have reusable logging across implementations and a one-time setup method for the system.

Answer:

I'll define a Notifier interface with a default log() method and a static setup() method. Implementations can focus only on the send() method.

Code:

```
interface Notifier {
    void send(String message);

    default void log(String message) {
        System.out.println("[LOG] " + message);
    }

    static void setup() {
        System.out.println("Notifier system initialized...");
    }
}

class PushNotifier implements Notifier {
    public void send(String message) {
        log(message);
        System.out.println("Sending PUSH notification: " + message);
    }
}

class EmailNotifier implements Notifier {
    public void send(String message) {
        log(message);
        System.out.println("Sending EMAIL notification: " + message);
    }
}

public class NotifierTest {
    public static void main(String[] args) {
        Notifier.setup();
        Notifier push = new PushNotifier();
        push.send("App update available");

        Notifier email = new EmailNotifier();
```

```
        email.send("Welcome to our platform");
    }
}
```

Reasoning to interviewer:

- The default log() method ensures consistent logging across all notifier types without duplicating code.
- The static setup() method provides system-wide initialization (e.g., database connections, configuration).
- Each notifier implementation only overrides send() with its own delivery logic.
- This design showcases how Java 8 default and static methods reduce boilerplate and centralize shared behavior.

Future scope:

- Extend with more notifiers like SMSNotifier.
- Add default error-handling logic in the interface.
- Integrate logging with external monitoring systems (e.g., ELK, Prometheus).

Scenario 37: Marketplace – sorting products by multiple fields

Each product has a category and price. The requirement is to sort items by category alphabetically and then by price within each category.

Answer:

I'll use Comparator.comparing on category, followed by thenComparing on price.

Code:

```
import java.util.*;

class ProductItem {
    String category;
    double price;

    public ProductItem(String category, double price) {
        this.category = category;
        this.price = price;
    }

    public String getCategory() { return category; }
    public double getPrice() { return price; }

    @Override
    public String toString() {
        return "ProductItem{category='" + category + "', price=" + price + "}";
    }
}

public class ProductSortDemo {
    public static void main(String[] args) {
        List<ProductItem> items = Arrays.asList(
            new ProductItem("Clothing", 1200),
            new ProductItem("Electronics", 800),
            new ProductItem("Clothing", 600),
            new ProductItem("Electronics", 1500)
        );

        items.sort(Comparator.comparing(ProductItem::getCategory)
                             .thenComparing(ProductItem::getPrice));

        items.forEach(System.out::println);
    }
}
```

Reasoning to interviewer:

- Comparator.comparing(ProductItem::getCategory) ensures primary sorting by category.
- thenComparing(ProductItem::getPrice) applies secondary sorting by price within each category.
- This gives a predictable order across multiple fields without writing nested if-else.
- The code is concise, readable, and leverages method references for clarity.

Future scope:

- Reverse the comparator for descending price within each category.
- Add tertiary sorting (e.g., by product name if category and price are the same).
- Externalize sorting criteria so that admin users can choose the order dynamically.

Scenario 38: Analytics system – running Runnable and Callable jobs with execution time logging

The system schedules two types of jobs: fire-and-forget (Runnable) and those that return results (Callable). The team needs a utility to handle both, measure execution time, and log the duration.

Answer:

I'll use method overloading: one runJob method that accepts a Runnable and another generic one that accepts a Callable<T>. Both methods will log the execution time, and the Callable version will return the result.

Code:

```
import java.util.concurrent.Callable;

public class JobRunner {

    public void runJob(Runnable job) {
        long start = System.currentTimeMillis();
        job.run();
        long end = System.currentTimeMillis();
        System.out.println("Runnable job completed in " + (end - start) + " ms");
    }

    public <T> T runJob(Callable<T> job) throws Exception {
        long start = System.currentTimeMillis();
        T result = job.call();
        long end = System.currentTimeMillis();
        System.out.println("Callable job completed in " + (end - start) + " ms");
        return result;
    }

    public static void main(String[] args) throws Exception {
        JobRunner runner = new JobRunner();

        // Runnable job
        runner.runJob(() -> System.out.println("Cleaning cache..."));

        // Callable job
        String data = runner.runJob(() -> {
            Thread.sleep(300);
            return "Report-123";
        });
        System.out.println("Result: " + data);
```

```
    }
}
```

Reasoning to interviewer:

- Overloading runJob lets the utility handle both Runnable and Callable jobs without forcing unnecessary returns.
- System.currentTimeMillis() is used for simple time measurement.
- In the Callable case, the result is returned so that calling code can consume it.
- This design provides flexibility while keeping the API intuitive.

Future scope:

- Replace System.currentTimeMillis() with System.nanoTime() for more precise timing.
- Extend support to asynchronous execution using ExecutorService.
- Add exception handling and logging frameworks for production use.

Scenario 39: Financial system – total salary of high earners

Each staff record has a name and annual income. Management wants the total salary of staff earning more than ₹40,00,000 to analyze executive payroll distribution.

Answer:

I'll filter staff above the threshold, then use mapToDouble to extract salaries and sum() to total them.

Code:

```
import java.util.*;

class Staff {
    String name;
    double annualIncome;

    public Staff(String name, double annualIncome) {
        this.name = name;
        this.annualIncome = annualIncome;
    }

    public double getAnnualIncome() {
        return annualIncome;
    }
}

public class HighEarnerDemo {
    public static void main(String[] args) {
        List<Staff> staffList = Arrays.asList(
            new Staff("Neha", 5200000),
            new Staff("Ravi", 3800000),
            new Staff("Ananya", 6100000)
        );

        double totalHighEarnings = staffList.stream()
            .filter(s -> s.getAnnualIncome() > 4000000)
            .mapToDouble(Staff::getAnnualIncome)
            .sum();

        System.out.println("Total income of staff earning above 40L: ₹" +
totalHighEarnings);
    }
}
```

Reasoning to interviewer:

- filter narrows down staff to only those above the 40L threshold.
- mapToDouble extracts annual income as a primitive double stream.
- sum() aggregates the incomes into a single total.
- This provides a clean one-liner calculation without manual loops.

Future scope:

- Extend to calculate average income of high earners with average().
- Use summaryStatistics() to get min, max, and count alongside the sum.
- Group staff into high vs normal earners using partitioningBy.

Scenario 40: User service – using Optional to avoid null checks

The service returns an Optional<User> when searching by email. The requirement is to log the user if present and return the display name, or return "Guest" if no user is found.

Answer:

I'll use ifPresent for logging, map to extract the display name, and orElse to provide a default value.

Code:

```
import java.util.*;

class User {
    String email;
    String displayName;

    User(String email, String displayName) {
        this.email = email;
        this.displayName = displayName;
    }

    String getDisplayName() { return displayName; }
}

public class OptionalUserDemo {
    public static void main(String[] args) {
        Optional<User> maybeUser = Optional.of(new User("a@x.com", "Akash"));
        // Optional<User> maybeUser = Optional.empty();

        maybeUser.ifPresent(u -> System.out.println("Found: " +
u.getDisplayName()));

        String name = maybeUser.map(User::getDisplayName).orElse("Guest");
        System.out.println("Hello, " + name);
    }
}
```

Reasoning to interviewer:

- ifPresent executes the block only when the user exists, avoiding null checks.
- map(User::getDisplayName) extracts the display name if present.
- orElse("Guest") provides a fallback for the missing case.

- This makes the code cleaner, safer, and more expressive compared to if (user != null).

Future scope:

- Use orElseGet if computing the default value is expensive.
- Chain additional operations like filter to validate the user before logging.
- Apply this pattern widely to replace defensive null-checks in service layers.

Scenario 41: Parsing product names – using constructor references

We need to convert product names from a CSV into Product instances. Instead of a lambda, use a constructor reference.

Answer:

I'll use Product::new as a constructor reference since it matches the expected signature in map.

Code:

```
import java.util.*;
import java.util.stream.*;

class Product {
    String name;
    Product(String name){ this.name = name; }

    @Override
    public String toString(){ return name; }
}

public class ProductConstructorRefDemo {
    public static void main(String[] args) {
        List<String> names = Arrays.asList("Laptop", "Phone", "Tablet");

        List<Product> products = names.stream()
            .map(Product::new)
            .collect(Collectors.toList());

        System.out.println(products);
    }
}
```

Reasoning to interviewer:

- A constructor reference Product::new is shorter and cleaner than (name) -> new Product(name).
- This improves readability when parsing or transforming data.
- It works because the constructor signature matches the functional interface (Function<String, Product>).

Future scope:

- Extend with overloaded constructors to handle more CSV fields (e.g., price, category).
- Combine with flatMap if parsing multi-column rows into multiple objects.
- Use constructor references in factories and mapping utilities for consistency.

Scenario 42: Courses – extracting unique tags

Each course has a list of tags. We need a flattened list of unique tags across all courses.

Answer:

I'll use flatMap to flatten nested lists of tags into one stream, and then distinct to remove duplicates.

Code:

```
import java.util.*;
import java.util.stream.*;

class Course {
    String title;
    List<String> tags;

    Course(String title, List<String> tags){
        this.title = title;
        this.tags = tags;
    }
}

public class UniqueTagsDemo {
    public static void main(String[] args) {
        List<Course> courses = Arrays.asList(
            new Course("Java", Arrays.asList("backend","java","oop")),
            new Course("Spring", Arrays.asList("java","backend","api"))
        );

        List<String> uniqueTags = courses.stream()
            .map(c -> c.tags)
            .flatMap(List::stream)
            .distinct()
            .collect(Collectors.toList());

        System.out.println(uniqueTags);
    }
}
```

Reasoning to interviewer:

- map(c -> c.tags) extracts the list of tags per course.
- flatMap(List::stream) flattens multiple lists into a single stream.
- distinct eliminates duplicates, leaving only unique tags.

- This is far more elegant than nested loops for building a unique tag set.

Future scope:

- Collect into a Set instead of List if uniqueness is the only concern.
- Sort the unique tags alphabetically for UI presentation.
- Use tagging analytics (frequency counts) by combining groupingBy with counting().

Scenario 43: Compute both min and max in one pass

We need to find both minimum and maximum values from a list of integers in a single traversal.

Answer:

I'll create a small MinMax holder class and use reduce. Each element updates the holder by combining min and max values.

Code:

```
import java.util.*;

class MinMax {
    final int min, max;

    MinMax(int min, int max) {
        this.min = min;
        this.max = max;
    }

    MinMax combine(int v) {
        return new MinMax(Math.min(min, v), Math.max(max, v));
    }

    static MinMax of(int v) {
        return new MinMax(v, v);
    }

    @Override
    public String toString() {
        return "min=" + min + ", max=" + max;
    }
}

public class MinMaxDemo {
    public static void main(String[] args) {
        int[] arr = {7, 2, 9, 4, 6};

        Optional<MinMax> mm = Arrays.stream(arr).boxed()
            .reduce(
                Optional.<MinMax>empty(),
                (opt, v) -> Optional.of(opt.map(m ->
m.combine(v)).orElse(MinMax.of(v))),
                (o1, o2) -> o1.isPresent() ? o1 : o2 // trivial combiner for sequential
            );
```

```
        System.out.println(mm.orElse(null));
    }
}
```

Reasoning to interviewer:

- A custom MinMax holder lets us compute both values in one pass.
- reduce combines elements iteratively into the holder.
- Using Optional avoids special handling for the first element.
- This avoids multiple traversals (like separate min() and max()).

Future scope:

- Replace with a mutable accumulator for simpler code.
- Use IntSummaryStatistics (via Collectors.summarizingInt) which also gives count and average.
- Parallelize with a proper combiner for large data sets.

Scenario 44: Summing millions of sales with parallel streams

We need to compute the total sales efficiently when dealing with millions of records.

Answer:

I'll use a parallel stream with an associative reduction operation. For numeric sums, mapToDouble().sum() is safe and efficient.

Code:

```
import java.util.*;

public class ParallelSalesDemo {
    public static void main(String[] args) {
        List<Double> sales = Arrays.asList(10.0, 20.0, 30.0, 40.0);

        double total = sales.parallelStream()
            .mapToDouble(Double::doubleValue)
            .sum(); // associative, safe in parallel

        System.out.println("Total: " + total);
    }
}
```

Reasoning to interviewer:

- Parallel streams split work across multiple threads.
- Summation is associative, so it's safe to compute in parallel.
- The result is accurate and typically faster for very large, CPU-bound datasets.
- For small data, overhead of parallelism outweighs benefits.

Future scope:

- Benchmark sequential vs parallel before adopting.
- Use custom thread pools via ForkJoinPool if tuning is required.
- Combine with collectors like summarizingDouble for richer stats.

Scenario 45: Customer segmentation – premium vs regular

We need to split customers into two buckets: premium (spent ≥ ₹10k) and regular (spent < ₹10k).

Answer:

I'll use Collectors.partitioningBy with the condition on lifetime spend.

Code:

```
import java.util.*;
import java.util.stream.*;

class Customer {
    String id;
    double lifetimeSpend;

    Customer(String id, double s) {
        this.id = id;
        this.lifetimeSpend = s;
    }

    @Override
    public String toString() {
        return id + " (₹" + lifetimeSpend + ")";
    }
}

public class PartitionCustomersDemo {
    public static void main(String[] args) {
        List<Customer> customers = Arrays.asList(
            new Customer("C1", 15000),
            new Customer("C2", 4000),
            new Customer("C3", 12000)
        );

        Map<Boolean, List<Customer>> buckets = customers.stream()
            .collect(Collectors.partitioningBy(c -> c.lifetimeSpend >= 10_000));

        System.out.println("Premium: " + buckets.get(true));
        System.out.println("Regular: " + buckets.get(false));
    }
}
```

Reasoning to interviewer:

- partitioningBy always creates two buckets based on a boolean condition.

- Customers with spend ≥ 10k fall into the true bucket (premium).
- Others automatically fall into the false bucket (regular).
- This avoids writing two separate filters or manual loops.

Future scope:

- Enhance to nested grouping by region or age group within premium/regular buckets.
- Replace List<Customer> with counts using partitioningBy(predicate, counting()).
- Use for feature flags, e.g., splitting users into “eligible” and “ineligible” categories.

Scenario 46: Immutable top 3 scores

We need a sorted list of the top 3 scores that cannot be modified afterwards.

Answer:

I'll sort scores in descending order, take the top 3, then wrap the result with collectingAndThen to make it unmodifiable.

Code:

```
import java.util.*;
import java.util.stream.*;

public class TopScoresDemo {
    public static void main(String[] args) {
        List<Integer> scores = Arrays.asList(75, 88, 91, 67, 99, 84);

        List<Integer> top3 = scores.stream()
            .sorted(Comparator.reverseOrder())
            .limit(3)
            .collect(Collectors.collectingAndThen(
                Collectors.toList(),
                Collections::unmodifiableList
            ));

        System.out.println(top3);

        // top3.add(100); // throws UnsupportedOperationException
    }
}
```

Reasoning to interviewer:

- sorted(Comparator.reverseOrder()) sorts scores high → low.
- limit(3) picks only the top 3.
- collectingAndThen allows us to transform the collected list into an unmodifiable version.
- This ensures integrity of results by preventing accidental modifications.

Future scope:

- Store the unmodifiable list in a cache or leaderboard system.
- Extend to top N scores dynamically using a parameter.

- Combine with map to wrap scores with player IDs for richer leaderboards.

Scenario 47: Build a CSV line of product names in alphabetical order

We need to join product names into a single CSV string after sorting them alphabetically.

Answer:

I'll create a custom collector using Collector.of with a StringJoiner as the mutable container.

Code:

```
import java.util.*;
import java.util.stream.*;
import java.util.StringJoiner;

public class CsvCollectorDemo {
    public static void main(String[] args) {
        List<String> names = Arrays.asList("Mouse", "Keyboard", "Laptop");

        Collector<String, StringJoiner, String> csvCollector =
            Collector.of(
                () -> new StringJoiner(","),          // supplier
                (sj, s) -> sj.add(s),                 // accumulator
                (sj1, sj2) -> sj1.merge(sj2),         // combiner
                StringJoiner::toString                // finisher
            );

        String csv = names.stream()
            .sorted()
            .collect(csvCollector);

        System.out.println(csv); // Keyboard,Laptop,Mouse
    }
}
```

Reasoning to interviewer:

- Collector.of gives full control over supplier, accumulator, combiner, and finisher.
- StringJoiner is a natural fit for building CSV strings efficiently.
- Sorting first ensures alphabetical order.

- This showcases knowledge of building custom collectors beyond the built-in ones.

Future scope:

- Extend collector to handle quoting/escaping for CSV compliance.
- Make delimiter configurable (e.g., semicolon, tab).
- Reuse collector for other reporting pipelines (e.g., export user emails).

Scenario 48: Sort employees by department (nulls last), then by name case-insensitive

Each employee has a department and name. We need to sort employees by department, putting null departments last, and then by name in a case-insensitive manner.

Answer:

I'll use Comparator.comparing with nullsLast for department, and then chain thenComparing with lowercase name comparison.

Code:

```
import java.util.*;

class Employee {
    String dept;
    String name;

    Employee(String d, String n) {
        this.dept = d;
        this.name = n;
    }

    String getDept() { return dept; }
    String getName() { return name; }

    @Override
    public String toString() {
        return dept + ":" + name;
    }
}

public class EmployeeSortDemo {
    public static void main(String[] args) {
        List<Employee> es = Arrays.asList(
            new Employee("Sales", "arjun"),
            new Employee(null, "Meera"),
            new Employee("Sales", "Bhavya"),
            new Employee("HR", "Zoya")
        );

        es.sort(Comparator.comparing(Employee::getDept,
                    Comparator.nullsLast(String::compareTo))
                .thenComparing(e -> e.getName().toLowerCase()));
```

```
        System.out.println(es);
    }
}
```

Reasoning to interviewer:

- Comparator.comparing(Employee::getDept, Comparator.nullsLast(...)) ensures departments are sorted but nulls come last.
- thenComparing applies secondary ordering on name.
- Using toLowerCase() makes the comparison case-insensitive.
- This combines multiple comparator helpers in a clean, declarative way.

Future scope:

- Add tertiary sorting (e.g., by employee ID).
- Make department ordering customizable (e.g., HR always before Sales).
- Externalize sort criteria into configuration for dynamic sorting.

Scenario 49: Order → Customer → Address → City with Optional

An order may have a customer, a customer may have an address, and the address may have a city. We need to get the city if available, otherwise return "N/A", without messy nested null checks.

Answer:

I'll wrap each hop (Order → Customer → Address → City) in Optional, then chain with flatMap/map, and finally use orElse to provide a default.

Code:

```
import java.util.*;

class Address {
    String city;
    Address(String c){ this.city = c; }
    String getCity(){ return city; }
}

class Customer {
    Optional<Address> address;
    Customer(Optional<Address> a){ this.address = a; }
    Optional<Address> getAddress(){ return address; }
}

class Order {
    Optional<Customer> customer;
    Order(Optional<Customer> c){ this.customer = c; }
    Optional<Customer> getCustomer(){ return customer; }
}

public class OrderCityDemo {
    public static void main(String[] args) {
        Order order = new Order(Optional.of(new Customer(Optional.of(new
Address("Pune")))));
        // Order order = new Order(Optional.empty());

        String city = Optional.of(order)
            .flatMap(Order::getCustomer)
            .flatMap(Customer::getAddress)
            .map(Address::getCity)
            .orElse("N/A");

        System.out.println("City: " + city);
    }
}
```

Reasoning to interviewer:

- Optional.of(order) wraps the root safely.
- Each flatMap unwraps one level (Customer, Address).
- map(Address::getCity) extracts the city if present.
- orElse("N/A") provides a clean default when anything along the chain is missing.
- This avoids deeply nested if (obj != null) checks.

Future scope:

- Replace orElse with orElseGet for lazy default computation.
- Add validation logic (e.g., non-empty city name).
- Extend the pattern for other optional chains like profile → settings → theme.

Scenario 50: User repository – Optional with logging and default

The repository returns Optional<User> when searching by email. We need to log if the user exists, then either display their name or show "Guest".

Answer:

I'll use ifPresent for logging, map to extract the display name, and orElse for the default value.

Code:

```
import java.util.*;

class User {
    String email, displayName;
    User(String email, String displayName){
        this.email = email;
        this.displayName = displayName;
    }
    String getDisplayName(){ return displayName; }
}

public class OptionalUserDemo {
    public static void main(String[] args) {
        Optional<User> maybeUser = Optional.of(new User("a@x.com","Akash"));
        // Optional<User> maybeUser = Optional.empty();

        maybeUser.ifPresent(u -> System.out.println("Found user: " +
u.getDisplayName()));

        String name = maybeUser.map(User::getDisplayName).orElse("Guest");
        System.out.println("Hello, " + name);
    }
}
```

Reasoning to interviewer:

- ifPresent logs when the user exists, replacing manual if (user != null) checks.
- map(User::getDisplayName) transforms the user into their display name.
- orElse("Guest") ensures a fallback string if the user is not present.
- This is concise, expressive, and eliminates risk of NullPointerException.

Future scope:

- Use orElseThrow when absence is exceptional instead of providing a default.
- Chain further operations (like trimming or uppercasing the display name).
- Wrap this logic in a service method for reuse across multiple controllers.

Scenario 51: Order → Customer → Address → City with Optional

We need to safely extract the city from an order where both Customer and Address may or may not exist, and return "N/A" if any part of the chain is missing.

Answer:

I'll wrap the root object in Optional, chain each access with flatMap/map, and finish with orElse for a safe default.

Code:

```
import java.util.*;

class Address {
    String city;
    Address(String c){ this.city=c; }
    String getCity(){ return city; }
}

class Customer {
    Optional<Address> address;
    Customer(Optional<Address> a){ this.address=a; }
    Optional<Address> getAddress(){ return address; }
}

class Order {
    Optional<Customer> customer;
    Order(Optional<Customer> c){ this.customer=c; }
    Optional<Customer> getCustomer(){ return customer; }
}

public class OrderCityDemo {
    public static void main(String[] args) {
        Order order = new Order(Optional.of(new Customer(Optional.of(new
Address("Pune")))));
        // Order order = new Order(Optional.empty());

        String city = Optional.of(order)
            .flatMap(Order::getCustomer)
            .flatMap(Customer::getAddress)
            .map(Address::getCity)
            .orElse("N/A");

        System.out.println("City: " + city);
    }
}
```

Reasoning to interviewer:

- Wrapping in Optional.of(order) allows us to safely start the chain.
- Each flatMap unwraps the next level (Customer, Address) if present.
- map(Address::getCity) extracts the city when the chain exists.
- orElse("N/A") provides a fallback when any step is absent.
- This replaces multiple nested null checks with a fluent, safe pipeline.

Future scope:

- Replace orElse with orElseGet for lazy evaluation of default values.
- Use orElseThrow if city absence should be treated as an error.
- Apply the same pattern for other deeply nested optional properties.

Scenario 52: Construct Product objects from CSV names

We need to create Product objects from a list of names parsed from a CSV file.

Answer:

I'll use a constructor reference (Product::new) instead of a lambda for object creation.

Code:

```
import java.util.*;
import java.util.stream.*;

class Product {
    String name;
    Product(String name){ this.name = name; }
    @Override public String toString(){ return name; }
}

public class ProductConstructorRefDemo {
    public static void main(String[] args) {
        List<String> names = Arrays.asList("Laptop", "Phone", "Tablet");

        List<Product> products = names.stream()
            .map(Product::new) // constructor reference
            .collect(Collectors.toList());

        System.out.println(products);
    }
}
```

Reasoning to interviewer:

- A constructor reference (Product::new) is cleaner and shorter than (name) -> new Product(name).
- It improves readability and clearly shows intent: mapping names to Product instances.
- Works because the Product(String) constructor matches the functional interface Function<String, Product>.

Future scope:

- Extend Product with more fields like price and category, then adapt constructor references accordingly.
- Use parsing logic to handle multi-column CSVs and build richer Product objects.
- Combine with flatMap if each line expands into multiple products.

Scenario 53: Courses – unique, sorted tag list

Each course contains a list of tags. We want a single, de-duplicated, alphabetically sorted list of all tags across courses.

Answer:

I'll stream the courses, map each to its tag list, flatten with flatMap, eliminate duplicates using distinct, and finally sorted before collecting.

Code:

```
import java.util.*;
import java.util.stream.*;

class Course {
    String title;
    List<String> tags;
    Course(String t, List<String> tags){ this.title = t; this.tags = tags; }
}

public class UniqueSortedTagsDemo {
    public static void main(String[] args) {
        List<Course> courses = Arrays.asList(
            new Course("Java", Arrays.asList("backend","java","oop")),
            new Course("Spring", Arrays.asList("java","backend","api"))
        );

        List<String> uniqueTags = courses.stream()
            .map(c -> c.tags)           // Stream<List<String>>
            .flatMap(List::stream)      // Stream<String>
            .distinct()                 // remove dupes
            .sorted()                   // alphabetical
            .collect(Collectors.toList());

        System.out.println(uniqueTags);  // [api, backend, java, oop]
    }
}
```

Reasoning to interviewer:

- map(c -> c.tags) pulls out the tag lists; flatMap(List::stream) flattens them into a single stream of strings.
- distinct removes repeated tags like "java" and "backend".
- sorted gives a clean, UI-friendly order.
- This reads naturally left-to-right and avoids manual set/list juggling.

Future scope:

- Collect straight into a TreeSet with Collectors.toCollection(TreeSet::new) to get uniqueness + sorting in one step.
- Add case-insensitive sorting by using sorted(String.CASE_INSENSITIVE_ORDER).
- Track tag frequency with Collectors.groupingBy(tag -> tag, counting()) to surface trending topics.

Scenario 54: Integers – compute min and max in one pass (custom reduce)

We want both min and max from a single stream traversal, without separate collectors.

Answer:

I'll use a tiny immutable holder MinMax and fold the stream with reduce. Each value updates the holder by combining its current min/max with the new element.

Code:

```
import java.util.*;
import java.util.stream.*;

class MinMax {
    final int min, max;
    MinMax(int min, int max){ this.min = min; this.max = max; }
    MinMax combine(int v){ return new MinMax(Math.min(min, v), Math.max(max,
v)); }
    static MinMax of(int v){ return new MinMax(v, v); }
    @Override public String toString(){ return "min=" + min + ", max=" + max; }
}

public class MinMaxReduceDemo {
    public static void main(String[] args) {
        List<Integer> nums = Arrays.asList(7, 2, 9, 4, 6);

        Optional<MinMax> mm = nums.stream()
            .reduce(
                Optional.<MinMax>empty(),
                (opt, v) -> Optional.of(opt.map(m ->
m.combine(v)).orElse(MinMax.of(v))),
                (o1, o2) -> o1.isPresent() ? o1 : o2 // trivial combiner for sequential use
            );

        System.out.println(mm.orElse(null)); // min=2, max=9
    }
}
```

Reasoning to interviewer:

- MinMax is the fold target; every element updates it via combine.
- Wrapping in Optional sidesteps a special "first element" case and keeps the reduce total order.

- This stays single-pass and avoids running min() and max() as two separate traversals.
- It demonstrates comfort with custom reductions beyond built-in collectors.

Future scope:

- Prefer IntSummaryStatistics (stream.mapToInt(...).summaryStatistics()) when you also want count/avg and a simpler API.
- For parallel streams, supply a real combiner: MinMax combine(MinMax a, MinMax b) and use reduce(identity, accumulator, combiner).
- If mutability is acceptable, use a mutable holder to simplify the accumulator (less object churn).

Scenario 55: Sales totals – using parallel streams safely

We need to compute the total sales from millions of records as quickly as possible.

Answer:

I'll use a parallel stream with an associative reduction (sum) to split work across cores.

Code:

```
import java.util.*;

public class ParallelSalesDemo {
    public static void main(String[] args) {
        List<Double> sales = Arrays.asList(10.0, 20.0, 30.0, 40.0);

        double total = sales.parallelStream()
            .mapToDouble(Double::doubleValue)
            .sum();

        System.out.println("Total sales: " + total);
    }
}
```

Reasoning to interviewer:

- parallelStream breaks the dataset into chunks and processes them in multiple threads.
- mapToDouble().sum() is stateless and associative, so it's safe for parallel execution.
- Parallelism makes sense here only if:
    - The dataset is very large (millions of sales).
    - The operation is CPU-bound rather than IO-bound.
    - There's no shared mutable state that could cause race conditions.

Future scope:

- Benchmark sequential vs parallel to confirm benefit in production.
- Use a custom ForkJoinPool to control thread usage instead of the common pool.
- Extend to richer stats (min, max, average) using summaryStatistics().

Scenario 56: Customer segmentation – premium vs regular

We need to separate customers into two groups: premium (spent ≥ ₹10,000) and regular.

Answer:

I'll use Collectors.partitioningBy to split the stream into two buckets in a single pass.

Code:

```
import java.util.*;
import java.util.stream.*;

class Customer {
    String id;
    double lifetimeSpend;

    Customer(String id, double s){ this.id = id; this.lifetimeSpend = s; }

    @Override
    public String toString() {
        return id + " (₹" + lifetimeSpend + ")";
    }
}

public class CustomerPartitionDemo {
    public static void main(String[] args) {
        List<Customer> customers = Arrays.asList(
            new Customer("C1", 15000),
            new Customer("C2", 4000),
            new Customer("C3", 12000)
        );

        Map<Boolean, List<Customer>> buckets = customers.stream()
            .collect(Collectors.partitioningBy(c -> c.lifetimeSpend >= 10_000));

        System.out.println("Premium: " + buckets.get(true));
        System.out.println("Regular: " + buckets.get(false));
    }
}
```

Reasoning to interviewer:

- partitioningBy is a specialized collector that splits elements into two lists based on a boolean predicate.

- Here, customers with lifetimeSpend ≥ 10,000 go into the true bucket, others into false.
- This avoids running two separate filters and ensures one linear scan.

Future scope:

- Replace List<Customer> with counts using partitioningBy(predicate, counting()) for quick stats.
- Extend to nested grouping (e.g., within premium/regular, group by region).
- Use this approach for other binary splits like active vs inactive, verified vs unverified.

Scenario 57: Scores – immutable top 3 list

We have a bunch of scores and want the top three, returned as a list that nobody can modify later.

Answer:

I'll sort in descending order, take the first three, and then wrap the collected list using collectingAndThen with Collections.unmodifiableList.

Code:

```
import java.util.*;
import java.util.stream.*;

public class Top3ImmutableDemo {
    public static void main(String[] args) {
        List<Integer> scores = Arrays.asList(75, 88, 91, 67, 99, 84);

        List<Integer> top3 = scores.stream()
            .sorted(Comparator.reverseOrder())
            .limit(3)
            .collect(Collectors.collectingAndThen(
                Collectors.toList(),
                Collections::unmodifiableList
            ));

        System.out.println(top3);
        // top3.add(100); // throws UnsupportedOperationException
    }
}
```

Reasoning to interviewer:

- Sorting high→low and limit(3) gives the top three without extra data structures.
- collectingAndThen lets me post-process the collected result into an unmodifiable view.
- This prevents accidental mutation later (useful for leaderboards and cached outputs).

Future scope:

- Parameterize "3" to a runtime N.

- If the list is huge, consider a bounded min-heap for O(n log k) instead of full sort.
- Expose the result via an API that returns unmodifiable collections by default.

Scenario 58: Products – CSV of names in alphabetical order (custom collector)

We need a CSV string of product names, sorted alphabetically, using a tiny custom collector.

Answer:

I'll build a Collector around StringJoiner via Collector.of(supplier, accumulator, combiner, finisher) and use it after sorting.

Code:

```
import java.util.*;
import java.util.stream.*;
import java.util.StringJoiner;

public class ProductCsvCollectorDemo {
    public static void main(String[] args) {
        List<String> names = Arrays.asList("Mouse", "Keyboard", "Laptop");

        Collector<String, StringJoiner, String> csvCollector =
            Collector.of(
                () -> new StringJoiner(","),   // supplier
                StringJoiner::add,             // accumulator
                StringJoiner::merge,           // combiner (for parallel)
                StringJoiner::toString         // finisher
            );

        String csv = names.stream()
            .sorted()
            .collect(csvCollector);

        System.out.println(csv); // Keyboard,Laptop,Mouse
    }
}
```

Reasoning to interviewer:

- StringJoiner is efficient for delimiter-joined strings.
- The custom collector expresses the full reduce pipeline cleanly, including a proper combiner.
- Sorting first guarantees deterministic alphabetical output.

Future scope:

- Make delimiter and quoting configurable for true CSV compliance.
- Swap in locale-aware sorting (Collator) when needed.
- Reuse the collector for exporting IDs/emails elsewhere.

Scenario 59: Employees – sort by department (nulls last) then name (case-insensitive)

We must order employees by department, putting null departments at the end, and then sort by name ignoring case.

Answer:

I'll chain comparators using Comparator.comparing with nullsLast for department, then thenComparing on a lowercase name view.

Code:

```
import java.util.*;

class Employee {
    String dept;
    String name;
    Employee(String d, String n){ this.dept = d; this.name = n; }
    String getDept(){ return dept; }
    String getName(){ return name; }
    @Override public String toString(){ return dept + ":" + name; }
}

public class EmployeeSortDemo {
    public static void main(String[] args) {
        List<Employee> es = Arrays.asList(
            new Employee("Sales","arjun"),
            new Employee(null,"Meera"),
            new Employee("Sales","Bhavya"),
            new Employee("HR","Zoya")
        );

        es.sort(
            Comparator.comparing(Employee::getDept,
Comparator.nullsLast(String::compareTo))
                    .thenComparing(e -> e.getName().toLowerCase())
        );

        System.out.println(es);
    }
}
```

Reasoning to interviewer:

- nullsLast ensures missing departments don’t break comparisons and are pushed to the end.
- thenComparing provides deterministic ordering within each department.
- Using a lowercase projection makes the name sort case-insensitive without altering the data.

Future scope:

- Add a tertiary comparator (e.g., by employee ID) for complete tie-breaking.
- Externalize sort preferences so admins can choose department priority or name order.
- For large lists, consider stable sorts and pagination after sorting.

Scenario 60: Order analytics – all key stats (min/max/avg/sum/count) in one pass

We're aggregating monetary amounts from orders and want min, max, average, sum, and count without multiple traversals.

Answer:

I'll use Collectors.summarizingDouble, which returns a DoubleSummaryStatistics. It computes all five metrics in a single pass over the stream.

Code:

```
import java.util.*;
import java.util.stream.*;

class Order {
    String id;
    double amount;
    Order(String id, double amount) { this.id = id; this.amount = amount; }
}

public class OrderStatsDemo {
    public static void main(String[] args) {
        List<Order> orders = Arrays.asList(
            new Order("O1", 1299.99),
            new Order("O2", 499.50),
            new Order("O3", 2399.00),
            new Order("O4", 799.00)
        );

        DoubleSummaryStatistics stats = orders.stream()
            .collect(Collectors.summarizingDouble(o -> o.amount));

        System.out.println("count = " + stats.getCount());
        System.out.println("sum   = " + stats.getSum());
        System.out.println("min   = " + stats.getMin());
        System.out.println("max   = " + stats.getMax());
        System.out.println("avg   = " + stats.getAverage());
    }
}
```

Reasoning to interviewer:

- summarizingDouble performs a single linear scan and gives min, max, sum, average, and count.

- Avoids multiple collectors (or multiple passes) and keeps the pipeline expressive.
- The returned DoubleSummaryStatistics is handy for passing around one object with all metrics.

Future scope:

- If amounts are currency-sensitive, switch to BigDecimal (with a custom collector) to control rounding.
- Add segment-wise stats, e.g., groupingBy(status, summarizingDouble(...)) for per-status dashboards.
- Persist stats snapshots to power historical trend charts.

Scenario 61: Customer segmentation – partitioning with downstream stats (counts/averages per bucket)

We need premium vs regular customers (threshold ≥ ₹10,000) and want counts and averages for each bucket in one pass.

Answer:

I'll use Collectors.partitioningBy(predicate, downstream) with a downstream summarizingDouble on lifetimeSpend. That gives count, average (and more) per bucket.

Code:

```
import java.util.*;
import java.util.stream.*;

class Customer {
    String id;
    double lifetimeSpend;
    Customer(String id, double s) { this.id = id; this.lifetimeSpend = s; }
}

public class CustomerPartitionStatsDemo {
    public static void main(String[] args) {
        List<Customer> customers = Arrays.asList(
            new Customer("C1", 15000),
            new Customer("C2", 4000),
            new Customer("C3", 12000),
            new Customer("C4", 9500),
            new Customer("C5", 33000)
        );

        Map<Boolean, DoubleSummaryStatistics> statsByBucket =
customers.stream()
            .collect(Collectors.partitioningBy(
                c -> c.lifetimeSpend >= 10_000,
                Collectors.summarizingDouble(c -> c.lifetimeSpend)
            ));

        DoubleSummaryStatistics premium = statsByBucket.get(true);
        DoubleSummaryStatistics regular = statsByBucket.get(false);

        System.out.printf("Premium  → count=%d, avg=%.2f, min=%.2f,
max=%.2f%n",
                premium.getCount(), premium.getAverage(), premium.getMin(),
premium.getMax());
```

```
        System.out.printf("Regular  → count=%d, avg=%.2f, min=%.2f,
max=%.2f%n",
                regular.getCount(), regular.getAverage(), regular.getMin(),
regular.getMax());
    }
}
```

Reasoning to interviewer:

- partitioningBy splits into exactly two buckets based on the predicate (premium vs regular).
- The downstream summarizingDouble computes count and average (plus min/max/sum) for each bucket in the same pass.
- This avoids separate scans for counts vs averages and keeps the code compact and intention-revealing.

Future scope:

- If you only need counts/avg, expose just those fields to callers (DTO) via collectingAndThen.
- Replace hardcoded ₹10k with a configurable threshold or tiered groupingBy (e.g., Silver/Gold/Platinum).
- For currency correctness, model spend as BigDecimal and provide a custom downstream aggregator.

Scenario 62: Org analytics – nested grouping (department → team → average salary)

Each employee belongs to a department and a team, and has a salary. We want a two-level report: for every department, show each team's average salary.

Answer:

I'll use nested groupingBy: first by department, then by team, with averagingDouble as the downstream collector.

Code:

```
import java.util.*;
import java.util.stream.*;

class Employee {
    final String dept;
    final String team;
    final double salary;

    Employee(String dept, String team, double salary) {
        this.dept = dept;
        this.team = team;
        this.salary = salary;
    }

    String getDept() { return dept; }
    String getTeam() { return team; }
    double getSalary() { return salary; }

    @Override public String toString() {
        return dept + "/" + team + " : " + salary;
    }
}

public class NestedGroupingDemo {
    public static void main(String[] args) {
        List<Employee> emps = Arrays.asList(
            new Employee("Engineering", "Platform", 28_00_000),
            new Employee("Engineering", "Platform", 31_50_000),
            new Employee("Engineering", "Data", 27_00_000),
            new Employee("Sales", "Enterprise", 22_00_000),
            new Employee("Sales", "SMB", 18_50_000)
        );

        Map<String, Map<String, Double>> avgByDeptTeam =
```

```
            emps.stream()
                .collect(Collectors.groupingBy(
                    Employee::getDept,
                    Collectors.groupingBy(
                        Employee::getTeam,
                        Collectors.averagingDouble(Employee::getSalary)
                    )
                ));

        System.out.println(avgByDeptTeam);
    }
}
```

Reasoning to interviewer:

- groupingBy(dept, groupingBy(team, averagingDouble)) mirrors the report shape exactly.
- The result is a Map<Dept, Map<Team, AvgSalary>>, which is easy to render in a table/tree.
- A single pass computes everything; no manual interim structures.

Future scope:

- Swap averagingDouble with summarizingDouble if you also want min/max/count per team.
- Order keys deterministically by collecting into LinkedHashMap or sorting after collection.
- Add a third level (e.g., role) by nesting one more groupingBy.

Scenario 63: Mapping with duplicates – toMap merge function + insertion order

You're aggregating cart line items by SKU. Multiple rows may have the same SKU, and you want total quantity per SKU while preserving encounter order.

Answer:

I'll use Collectors.toMap with a merge function (Integer::sum) for duplicate keys, and LinkedHashMap::new as the map supplier to keep insertion order.

Code:

```
import java.util.*;
import java.util.stream.*;

class LineItem {
    final String sku;
    final int qty;

    LineItem(String sku, int qty) {
        this.sku = sku; this.qty = qty;
    }

    String getSku() { return sku; }
    int getQty() { return qty; }
    @Override public String toString(){ return sku + " x" + qty; }
}

public class ToMapMergeDemo {
    public static void main(String[] args) {
        List<LineItem> items = Arrays.asList(
            new LineItem("KB-101", 1),
            new LineItem("MS-200", 2),
            new LineItem("KB-101", 3),   // duplicate SKU
            new LineItem("HD-500", 1)
        );

        Map<String, Integer> qtyBySku = items.stream()
            .collect(Collectors.toMap(
                LineItem::getSku,           // key
                LineItem::getQty,           // value
                Integer::sum,               // merge duplicates
                LinkedHashMap::new          // preserve insertion order
            ));

        System.out.println(qtyBySku); // {KB-101=4, MS-200=2, HD-500=1}
```

```
    }
}
```

Reasoning to interviewer:

- toMap(keyFn, valFn, mergeFn, mapSupplier) is the “full control” overload.
- Integer::sum cleanly resolves duplicate keys by adding quantities.
- LinkedHashMap keeps first-seen key order, useful for UI or reporting where order matters.

Future scope:

- If you need the latest price per SKU, change the merge to “keep newer” (e.g., (a,b) -> b).
- Validate SKUs before aggregation; drop invalids with a filter.
- For large inputs, consider groupingBy then summingInt if you don’t care about order.

Scenario 64: Money totals – use BigDecimal (and rounding) for currency-safe sums

We're aggregating monetary amounts and need accuracy (no floating-point drift) and proper rounding for display or invoicing.

Answer:

I'll model amounts with BigDecimal, sum with groupingBy(currency) and reducing(BigDecimal::add), and round only at the edges with a RoundingMode.

Code:

```
import java.math.BigDecimal;
import java.math.RoundingMode;
import java.util.*;
import java.util.stream.*;

class Txn {
    final String currency;    // e.g., "INR", "USD"
    final BigDecimal amount;  // store as BigDecimal

    Txn(String currency, BigDecimal amount) {
        this.currency = currency;
        this.amount = amount;
    }
    String getCurrency() { return currency; }
    BigDecimal getAmount() { return amount; }
    @Override public String toString() { return currency + " " + amount; }
}

public class MoneyTotalsDemo {
    public static void main(String[] args) {
        List<Txn> txns = Arrays.asList(
            new Txn("INR", new BigDecimal("199.99")),
            new Txn("INR", new BigDecimal("850.10")),
            new Txn("USD", new BigDecimal("12.30")),
            new Txn("USD", new BigDecimal("0.70")),
            new Txn("INR", new BigDecimal("50.015")) // example with 3 decimals
        );

        Map<String, BigDecimal> totalByCurrency = txns.stream()
            .collect(Collectors.groupingBy(
                Txn::getCurrency,
                Collectors.mapping(
                    Txn::getAmount,
                    Collectors.reducing(BigDecimal.ZERO, BigDecimal::add)
```

```
                )
            ));

        // Round for presentation (business rule: 2 decimals, HALF_UP)
        Map<String, BigDecimal> roundedForDisplay =
totalByCurrency.entrySet().stream()
            .collect(Collectors.toMap(
                Map.Entry::getKey,
                e -> e.getValue().setScale(2, RoundingMode.HALF_UP),
                (a,b) -> a, LinkedHashMap::new
            ));

        System.out.println("Accurate totals: " + totalByCurrency);
        System.out.println("Rounded for display: " + roundedForDisplay);
    }
}
```

Reasoning to interviewer:

- Currency math with double can accumulate rounding error; BigDecimal keeps it exact.
- groupingBy + reducing(BigDecimal::add) gives per-currency totals in one pass.
- Rounding is postponed to the boundary (display/export) with a clear rule (HALF_UP, 2 decimals).
- If a currency requires 0/3 decimals, the same pattern works by changing scale per currency.

Future scope:

- Store currency minor units (e.g., paise/cents) as long for even faster arithmetic.
- Use Currency/ISO metadata to pick the correct scale per currency automatically.
- Add tax/discount pipelines with BigDecimal arithmetic and consistent MathContext.

Scenario 65: High-volume counting – groupingByConcurrent with parallelStream

You're processing millions of log entries and want a fast "hits per endpoint" report. Throughput matters more than ordering.

Answer:

I'll use a parallel stream with Collectors.groupingByConcurrent and a concurrent-friendly downstream (counting). This collects into a ConcurrentMap efficiently.

Code:

```
import java.util.*;
import java.util.concurrent.*;
import java.util.stream.*;

class LogEntry {
    final String endpoint; // e.g., "/api/orders", "/health"
    final int status;      // e.g., 200, 500
    LogEntry(String endpoint, int status) { this.endpoint = endpoint; this.status =
status; }
    String getEndpoint() { return endpoint; }
    int getStatus() { return status; }
}

public class ConcurrentGroupingDemo {
    public static void main(String[] args) {
        List<LogEntry> logs = Arrays.asList(
            new LogEntry("/api/orders", 200),
            new LogEntry("/api/orders", 200),
            new LogEntry("/api/users",  200),
            new LogEntry("/api/orders", 500),
            new LogEntry("/health",     200)
            // … imagine millions more
        );

        ConcurrentMap<String, Long> hitsByEndpoint = logs.parallelStream()
            .collect(Collectors.groupingByConcurrent(
                LogEntry::getEndpoint,
                Collectors.counting()
            ));

        System.out.println(hitsByEndpoint);
    }
}
```

Reasoning to interviewer:

- groupingByConcurrent uses a ConcurrentMap and is designed for parallel reductions.
- counting is stateless and associative, so it scales well in parallel.
- This is ideal when key contention isn't extreme and the cost of per-element work is small.
- Trade-off: result ordering isn't guaranteed; if you need order, post-process (e.g., sort entries).

Future scope:

- For very hot keys, a custom downstream using LongAdder can reduce contention.
- Extend to nested concurrent grouping (e.g., endpoint → status → counts).
- After collecting, compute top N endpoints with a bounded priority queue or stream sort.

Scenario 66: Compute min and max together (one pass with teeing)

You've got a bunch of numbers and you want both min and max without multiple traversals, and you'd like the code to read cleanly.

Answer:

I'll use Collectors.teeing (Java 12+) to run minBy and maxBy in parallel and merge their results into a tiny holder.

Code:

```
import java.util.*;
import java.util.stream.*;

class MinMax {
    final int min, max;
    MinMax(int min, int max) { this.min = min; this.max = max; }
    @Override public String toString() { return "min=" + min + ", max=" + max; }
}

public class TeeingMinMaxDemo {
    public static void main(String[] args) {
        List<Integer> nums = Arrays.asList(7, 2, 9, 4, 6);

        Optional<MinMax> result = nums.stream()
            .collect(Collectors.teeing(
                Collectors.minBy(Integer::compareTo),
                Collectors.maxBy(Integer::compareTo),
                (minOpt, maxOpt) -> (minOpt.isPresent() && maxOpt.isPresent())
                        ? Optional.of(new MinMax(minOpt.get(), maxOpt.get()))
                        : Optional.empty()
            ));

        System.out.println(result.orElse(null)); // min=2, max=9
    }
}
```

Reasoning to interviewer:

- teeing lets me compose two collectors (minBy, maxBy) in one pass and combine their outputs.
- Wrapping the final value in Optional keeps the empty-list case safe.
- The code says exactly what we're doing—find min and max—without custom reducers.

Future scope:

- Swap in summingDouble and counting to compute an average via teeing.
- If you’re on Java 8/11, fall back to summarizingInt/Double or a small custom reduce holder.
- For parallel workloads, teeing still works cleanly with the stream’s parallelism.

Scenario 67: CSV parsing — Optional + flatMap to drop bad rows

You're reading product rows from a CSV. Some lines are malformed. You want to parse into Product objects and silently skip invalid lines.

Answer:

I'll parse each line into an Optional<Product>. In the stream, I'll map to those optionals and use flatMap(Optional::stream) to keep only the valid ones.

Code:

```
import java.math.BigDecimal;
import java.util.*;
import java.util.stream.*;

class Product {
    final String name;
    final BigDecimal price;
    Product(String name, BigDecimal price) { this.name = name; this.price = price; }
    @Override public String toString() { return name + " @ " + price; }
}

public class OptionalFlatMapCsvDemo {

    static Optional<Product> parseRow(String row) {
        String[] parts = row.split(",", -1); // keep empties
        if (parts.length != 2) return Optional.empty();

        String name = parts[0].trim();
        String priceStr = parts[1].trim();
        if (name.isEmpty() || priceStr.isEmpty()) return Optional.empty();

        try {
            BigDecimal price = new BigDecimal(priceStr);
            return Optional.of(new Product(name, price));
        } catch (NumberFormatException ex) {
            return Optional.empty();
        }
    }

    public static void main(String[] args) {
        List<String> rows = Arrays.asList(
            "Laptop,799.99",
            "Phone,599.5",
            "BadRow",
            "Tablet,",          // missing price
            "  Headphones , 199.00 "
```

```
        );

        List<Product> products = rows.stream()
            .map(OptionalFlatMapCsvDemo::parseRow)  //
Stream<Optional<Product>>
            .flatMap(Optional::stream)              // Java 9+: keep only present
            // .filter(Optional::isPresent).map(Optional::get) // Java 8 alternative
            .collect(Collectors.toList());

        System.out.println(products);
    }
}
```

Reasoning to interviewer:

- The parser expresses success/failure via Optional, no exceptions leaked into the pipeline.
- flatMap(Optional::stream) is a neat “keep valid, drop invalid” pattern.
- Using BigDecimal for price avoids floating-point drift and is CSV-friendly.

Future scope:

- Enrich parsing with a proper CSV library (quotes, delimiters, escapes).
- Accumulate errors to an audit log while still returning all valid products.
- Add schema validation (min price, allowed categories) before constructing Product.

Scenario 68: Catalog – top N items per category (keep top 3 per group)

Each item has a category and a score (think rating, sales, or popularity). We want, for every category, the top 3 items by score.

Answer:

I'll group by category and use a downstream collector that maintains a bounded min-heap (priority queue). The heap keeps only the top N items. At the end, I'll convert it to a descending list.

Code:

```
import java.util.*;
import java.util.stream.*;

class Item {
    final String category;
    final String name;
    final int score; // higher is better

    Item(String category, String name, int score) {
        this.category = category; this.name = name; this.score = score;
    }
    String getCategory(){ return category; }
    int getScore(){ return score; }
    @Override public String toString(){ return name + "(" + score + ")"; }
}

public class TopNPerGroupDemo {

    // Generic "top N" downstream collector using a min-heap
    public static <T> Collector<T, PriorityQueue<T>, List<T>> topN(int n,
Comparator<? super T> cmp) {
        return Collector.of(
            () -> new PriorityQueue<>(cmp),               // min-heap by cmp
            (pq, t) -> { pq.offer(t); if (pq.size() > n) pq.poll(); },
            (left, right) -> {                            // merge two heaps
                for (T t : right) { left.offer(t); if (left.size() > n) left.poll(); }
                return left;
            },
            pq -> {                                       // finish: highest first
                List<T> out = new ArrayList<>(pq);
                out.sort(cmp.reversed());
                return out;
            }
```

```
        );
    }

    public static void main(String[] args) {
        List<Item> items = Arrays.asList(
            new Item("Books", "DSA Handbook", 92),
            new Item("Books", "System Design 101", 88),
            new Item("Books", "Clean Code", 95),
            new Item("Books", "Refactoring", 90),
            new Item("Games", "Speed Racer", 83),
            new Item("Games", "Dragon Quest", 96),
            new Item("Games", "Puzzle Pro", 89),
            new Item("Games", "Zen Garden", 85)
        );

        Map<String, List<Item>> top3ByCategory = items.stream()
            .collect(Collectors.groupingBy(
                Item::getCategory,
                topN(3, Comparator.comparingInt(Item::getScore)) // keep top 3 by
score
            ));

        top3ByCategory.forEach((cat, top) ->
            System.out.println(cat + " → " + top)
        );
    }
}
```

Reasoning to interviewer:

- The bounded min-heap discards the smallest element when size exceeds N, so we retain only the top N.
- Using it as a downstream collector inside groupingBy keeps everything single-pass.
- The finisher sorts the retained N items in descending order for clean display.

Future scope:

- Parameterize N per category (e.g., top 5 in Books, top 3 in Games).
- Break ties with a secondary comparator (e.g., score then name).
- If the groups are huge, consider streaming from a DB with window functions and push down the “top N” per partition.

Scenario 69: People directory – locale-aware, case/diacritic-insensitive sorting

Names can have accents and mixed case (e.g., “Álvaro”, “aarav”, “Émile”). We need human-friendly ordering based on locale rules, ignoring case and accents.

Answer:

I’ll use java.text.Collator for the target locale, set strength to PRIMARY (ignores case/diacritics), and sort with it.

Code:

```
import java.text.Collator;
import java.util.*;
import java.util.stream.*;

public class LocaleAwareSortDemo {
    public static void main(String[] args) {
        List<String> names = Arrays.asList(
            "Åke", "Zoë", "Émile", "Søren", "Álvaro", "aarav", "Aarav", "alvaro"
        );

        // Pick a locale that matches your audience; here we show Spanish-style
rules:
        Collator coll = Collator.getInstance(Locale.forLanguageTag("es"));
        coll.setStrength(Collator.PRIMARY); // ignore case and accents

        List<String> sorted = names.stream()
            .sorted(coll::compare)
            .collect(Collectors.toList());

        System.out.println(sorted);
    }
}
```

Reasoning to interviewer:

- Collator captures locale-specific collation rules (unlike plain String::compareTo).
- PRIMARY strength treats “Álvaro” and “alvaro” as equivalent for ordering, giving intuitive lists.
- Using coll::compare in sorted keeps the stream pipeline clean and expressive.

Future scope:

- Choose locale dynamically (user preference) and allow toggling strength (PRIMARY vs SECONDARY/TERTIARY).
- For complex objects, use Comparator.comparing(Person::getName, coll).
- After sorting, deduplicate near-equivalents by normalizing (NFKD) if needed for search indexes.